

REPIF

PVC Holder sprint

Maksym Bylym

1TPIFI

Table of Content

1.Introduction	3
2.Preparation and Marking	4
3. Drilling and Cutting the Openings	5
3.1. Standard Round Holes.....	5
3.2. Creating the Square Opening.....	5
4. Assembly of the Structure	6
5. Installation of Raspberry Pi and Display.....	6
5.1. Mounting the Raspberry Pi.....	6
5.2. Mounting the Display	6
5.3. Testing.....	6
6. Conclusion.....	6

1.Introduction

The goal of this project was to create a physical structure based on provided technical drawings and to install electronic components — specifically a Raspberry Pi board and a display. Over the course of one month, the work included marking according to the given diagrams, drilling and shaping openings, correcting small inaccuracies, assembling the parts, and mounting the electronics.

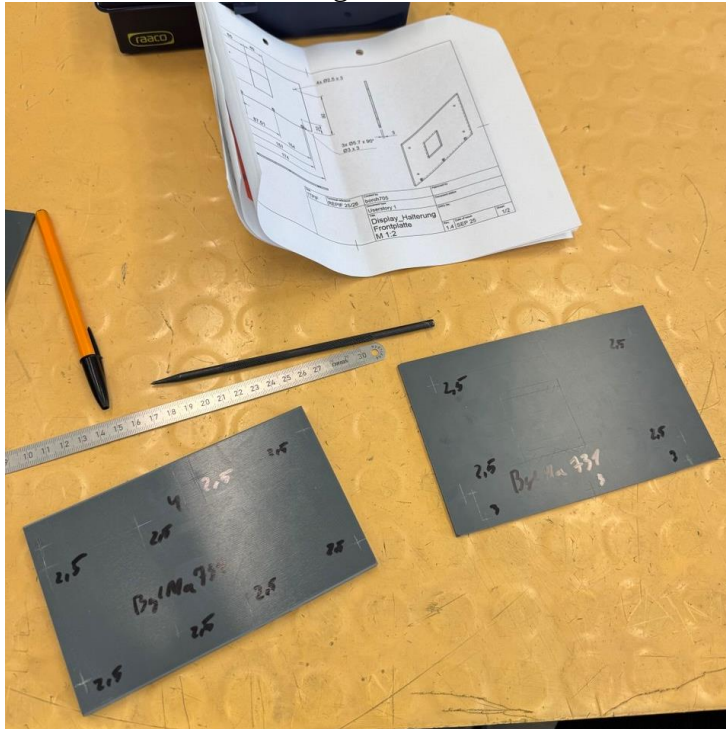
2.Preparation and Marking

(First Two Weeks)

At the beginning of the project, diagrams were provided indicating where to draw lines and where openings needed to be drilled or cut.

During the first two weeks, the following tasks were performed:

- Transferring the diagrams onto the boards and marking all required outlines.
- Repeatedly checking the markings to ensure accuracy before starting any cutting or drilling.
- Identifying and fixing one mistake: the outline of a square was shifted by 5 mm. This error was noticed before drilling and was corrected on time.



Picture1. The finished plates with the finalized markings

3. Drilling and Cutting the Openings

(3rd week)

After confirming that all markings were accurate, the technical work began.

3.1. Standard Round Holes

- Drilling was performed using a drilling machine.
- Standard drill bits of the required diameter were used.
- All holes were drilled according to the markings.



Picture2.The hole-drilling process

3.2. Creating the Square Opening

- Several large holes were drilled inside the marked square.
- The remaining material was removed manually.
- A file was used to refine the edges and create a smooth, clean square.



• Picture3. The drilled square



Picture 4. The polishing process

4. Assembly of the Structure

Once the openings were completed:

- The two plates were joined together using screws.
- This created the base for installing the electronic components.

5. Installation of Raspberry Pi and Display

5.1. Mounting the Raspberry Pi

- The Raspberry Pi board was attached to one of the plates using screws.
- The position matched the requirements of the design.



• Picture5. Mounted Raspberry PI

5.2. Mounting the Display

- The display was mounted on the opposite side of the structure.
- All necessary cables were connected between the display and the Raspberry Pi.



• Picture6. Mounted screen

5.3. Testing

After connecting all components, the system was tested — both the Raspberry Pi and the display worked correctly.

6. Conclusion

Throughout the month, every stage of the project was completed step by step: from marking and verifying measurements to drilling, assembling, and installing electronic components. Despite a small initial marking error, it was corrected early, and the final construction met all project requirements.

Hardware Assembly Documentation

REPIF

Maksym Bylym

Sprint: 2

PCB assembly, soldering, sensors integration

Table of content

Hardware Assembly Documentation	1
List of materials	3
Introduction	4
Soldering	5
Resistors	5
Other Components	7
Verification of All Components	9
Mounting and Fixation of the PCB	10

List of materials

Printed Circuit Board	Unassembled board intended for sensor and Raspberry Pi interface
Ribbon Cable	Used for connection between Raspberry Pi GPIO and the PCB
Screw Terminal Blocks	Used for power input/output and external connections
Resistors	Used for signal conditioning and current limiting
Integrated Circuits (ICs)	Used as part of signal processing or control circuitry
Pin Headers	Short pin header segments
JST-Type Connectors	Used for modular sensor connections
JST-Type Connectors	Used for internal electrical connections
Small Discrete Components	Used for internal electrical connections
Anti-Static Packaging	Used for safe storage and transport



Photo 1: Overview of components before assembly

Introduction

The objective of Sprint 2 is based on User Story A2 – Printed Circuit Board.

As the operator of the Portable Indoor Feedback station, the goal of this sprint is to manufacture and assemble a printed circuit board (PCB) that serves as an interface between:

- the Raspberry Pi,
- the connected sensors,
- and the actuators.

This printed circuit board allows the sensors to be neatly integrated into the station in order to record and process measured values in a structured and reliable way.

The sprint also requires:

- professional mounting and soldering of components,
- mechanical fixation of the PCB onto a PVC base plate,
- and proper technical documentation of the work steps.

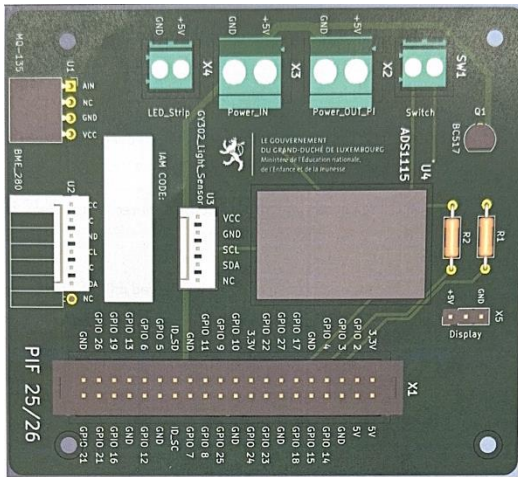


Photo 2: PCB layout – top view

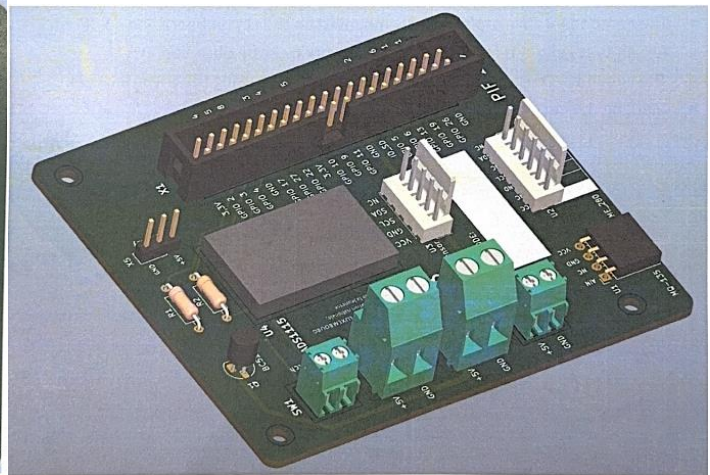


Photo 3: Assembled PCB – perspective view

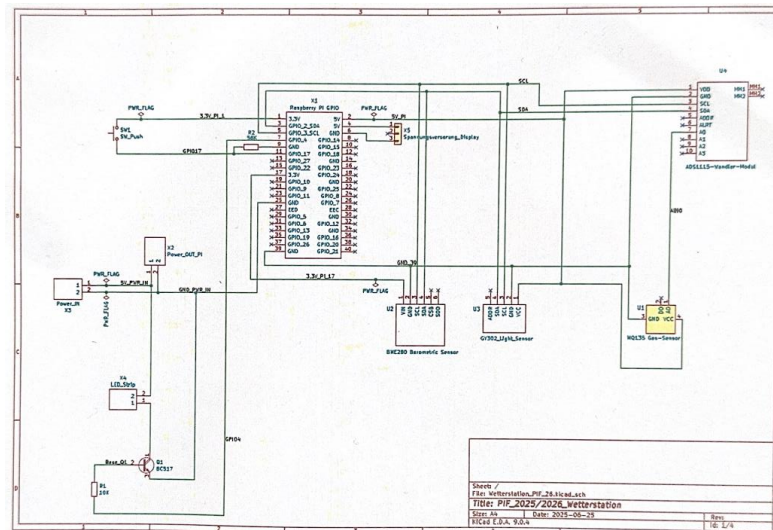


Photo 4: PCB with mounted connectors

Soldering Resistors

At the beginning of the soldering process, the two resistors required for the circuit were selected. Since the resistors had different nominal values (10 k and 56 k), their resistance values were measured using a digital multimeter in order to correctly identify each component.



Photo 5: Through-hole resistors before soldering

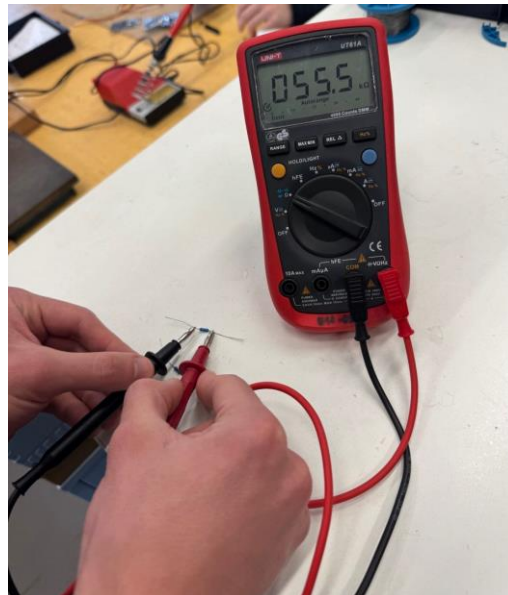


Photo 6: Measurement of resistor values using a digital multimeter

After confirming the resistance values, each resistor was placed into its designated position on the printed circuit board according to the PCB layout and schematic.

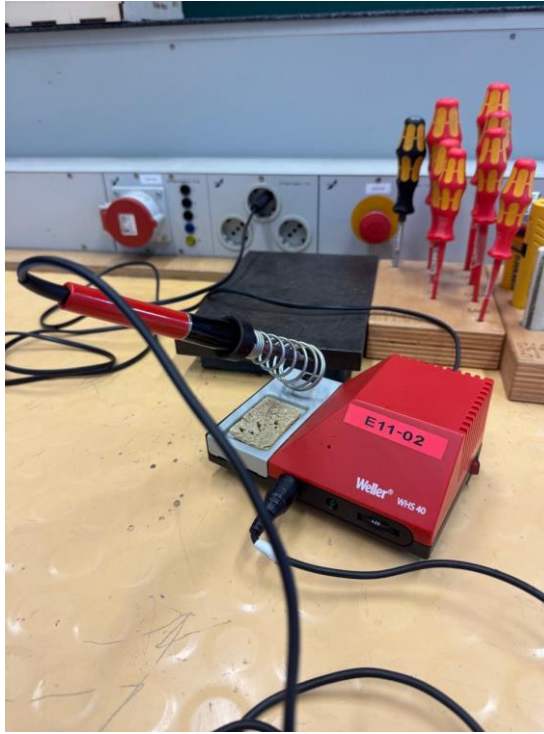


Photo 7: Soldering station and tools used during the soldering process

Once the soldering iron reached the appropriate operating temperature, the resistors were soldered onto the circuit board. During the initial soldering attempts, the solder joints were not optimal, as excessive solder was applied, resulting in large solder blobs.

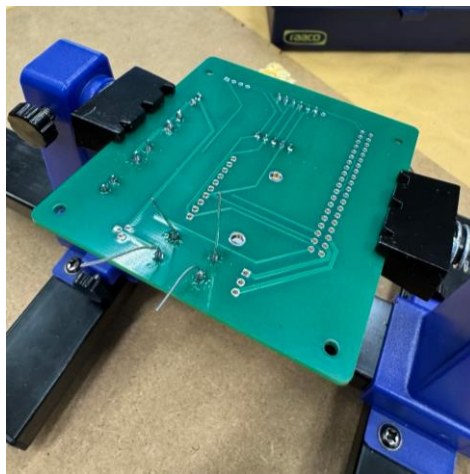


Photo 8: Underside of the PCB during resistor soldering

To correct this, the solder joints were reheated and excess solder was removed to improve the quality and cleanliness of the connections. After successful soldering, the protruding resistor leads on the underside of the PCB were trimmed using side cutters.



Photo 9: Trimming excess resistor leads after soldering

This process resulted in securely mounted resistors with clean and electrically reliable solder joints.



Photo 10: Printed circuit board fixed in a PCB holder during resistor soldering

Other Components

After completing the soldering of the resistors, the soldering process was continued with the remaining electronic components visible on the printed circuit board.

Based on the provided photos, the following components were soldered during this step:

- screw terminal blocks (2-pin) for power input, power output, LED strip, and switch connection,
- pin header connectors for the Raspberry Pi GPIO interface,
- JST-type connectors for sensor modules (MQ-135, BME280, and light sensor),
- additional pin headers required for module and signal connections.

During the soldering process, a consistent and controlled technique was applied. First, one outer pin of the component was soldered to fix its position. The alignment of the component was then checked to ensure correct placement on the PCB. After confirming proper positioning, the opposite outer pin was soldered. Once the component was correctly aligned and secured, the remaining pins were soldered.

This method helped to ensure accurate component placement, mechanical stability, and reliable electrical connections across the board.

REPIF – A2 Printed circuit board

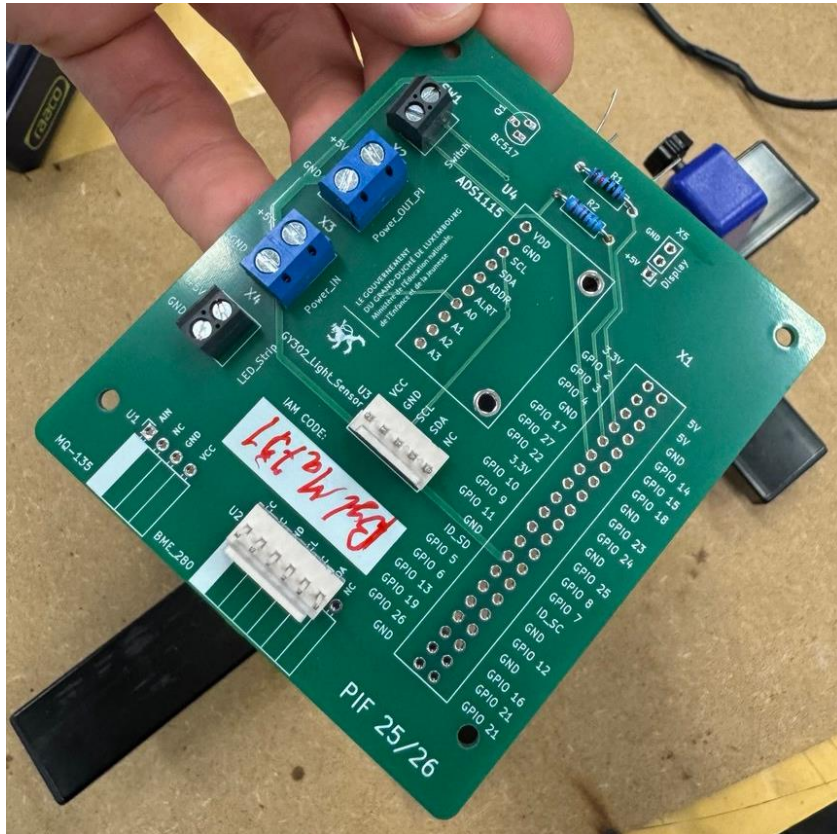


Photo 11: Soldering of connectors and terminal blocks on the PCB (top view)

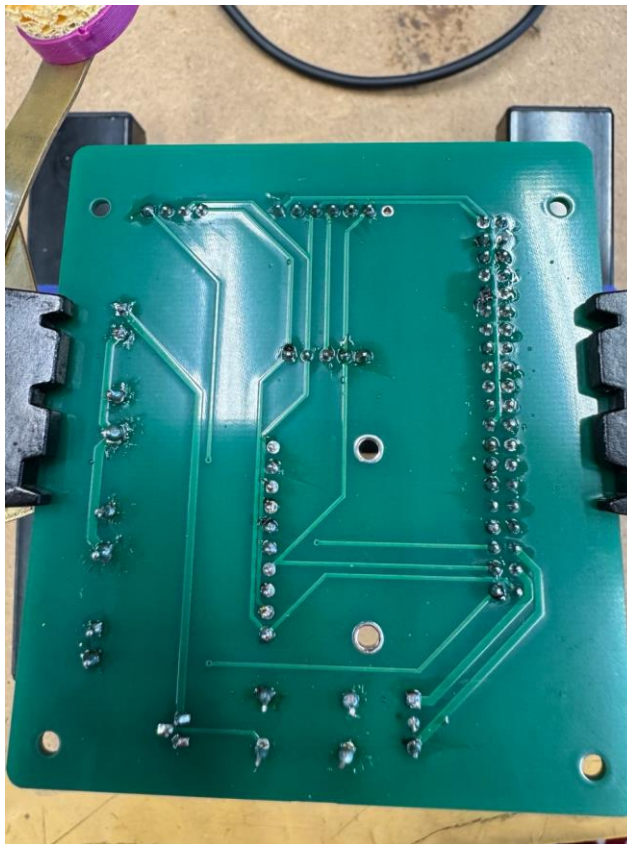


Photo 12: Underside of the PCB showing solder joints of connectors and headers

Verification of All Components

After completing the soldering of all main components, a full verification of the printed circuit board was carried out. During this step, all solder joints were visually inspected to ensure that:

- each component was placed in the correct position,
- all pins were properly soldered,
- no cold solder joints or short circuits were present,
- and the overall soldering quality was acceptable.

In addition to the verification process, a sensor intended for the next sprint was soldered onto the PCB. This sensor, located in the center of the board and identifiable by its blue module, was added in advance to prepare the hardware for future development steps.

The inspection was performed both before and after soldering the additional sensor, allowing a direct comparison of the PCB state and confirming that the integration did not affect existing connections.

This step ensured that the circuit board was electrically and mechanically ready for further assembly and integration in the following sprint.

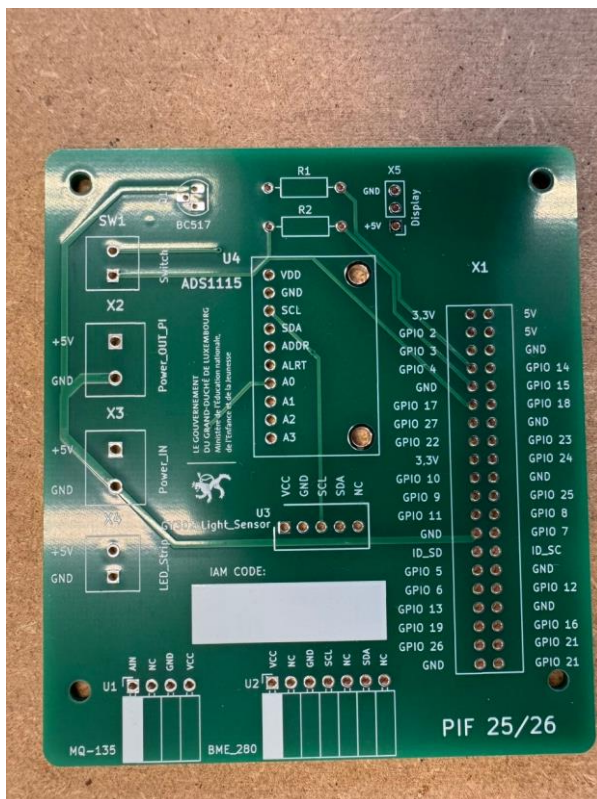


Photo 13: PCB before final component verification

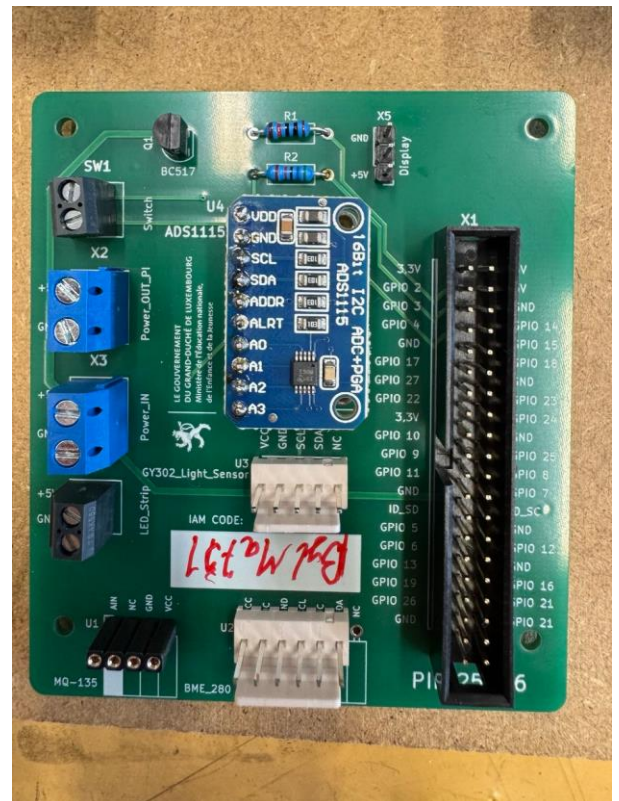


Photo 14: PCB after verification and soldering of the additional sensor

Mounting and Fixation of the PCB

After verifying that all components were correctly soldered and properly connected, the printed circuit board was mechanically mounted onto the base plate.

The PCB was fixed to the plate using four screws, ensuring stable and secure positioning. This mounting method provides sufficient mechanical support and prevents movement or stress on the solder joints during operation.

Once the board was fixed in place, all required connections were established. This included connecting the PCB to the Raspberry Pi using the ribbon cable, as well as connecting the necessary cables and connectors for power supply and sensor interfaces.

This final step completed the hardware assembly for the sprint and prepared the system for further testing and integration in the next development phase.

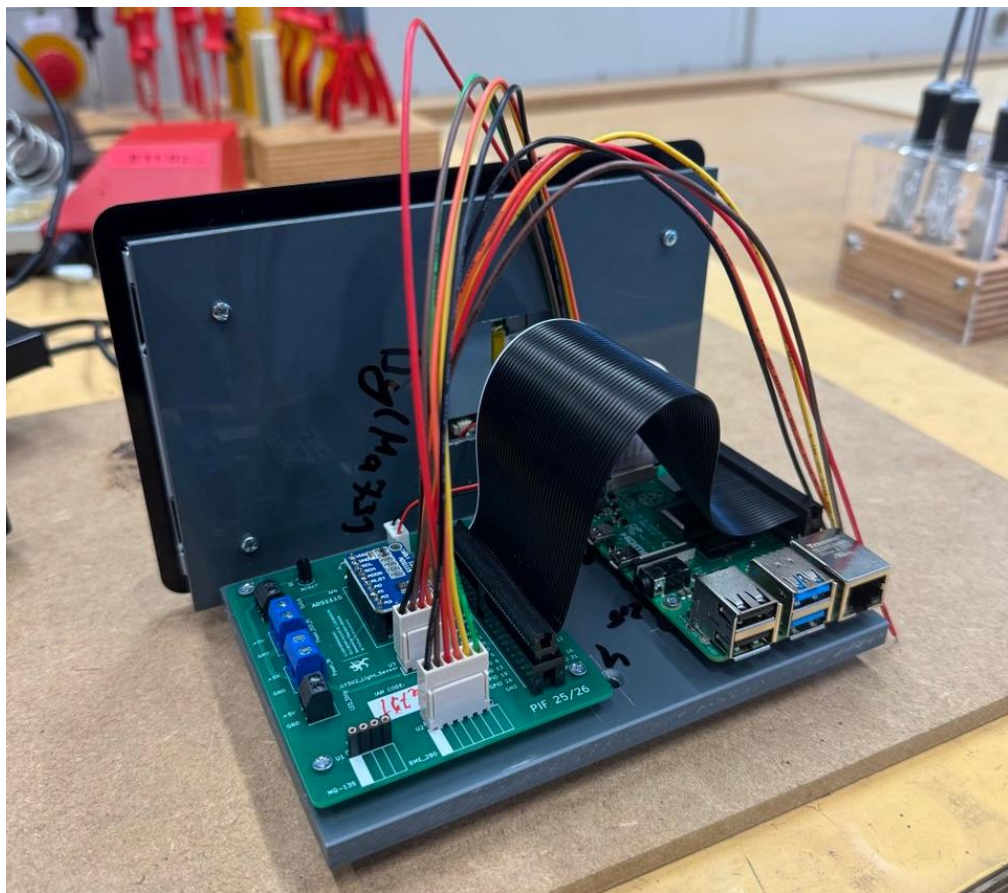


Photo 15: Printed circuit board mounted on the base plate and connected to the Raspberry Pi

REPIF - Power Supply

Maksym Bylym

Table of content

Introduction	3
Materials.....	3
Process	3

Introduction

The goal of this task was to establish a functional power supply for a Portable Indoor Feedback station.

Based on the given work order, the power cable had to be modified so the station could be put into operation.

This included cutting the cable, separating and insulating the internal conductors, and preparing them for connection.

Special attention was required to correctly handle the shielding and internal wires.

All connections had to be electrically safe and mechanically stable.

The result is a prepared and insulated power cable ready for further integration into the system.

Materials

Tools and Materials Used

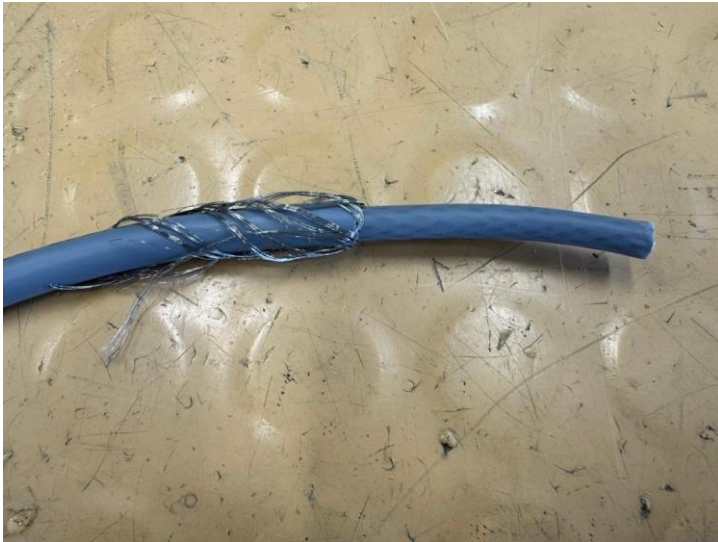
- Wire cutters (pliers)
- Automatic wire stripper
- Cable knife
- Soldering iron
- Solder wire
- Power supply cable



Process

First, I cut the cable into two parts using wire cutters.

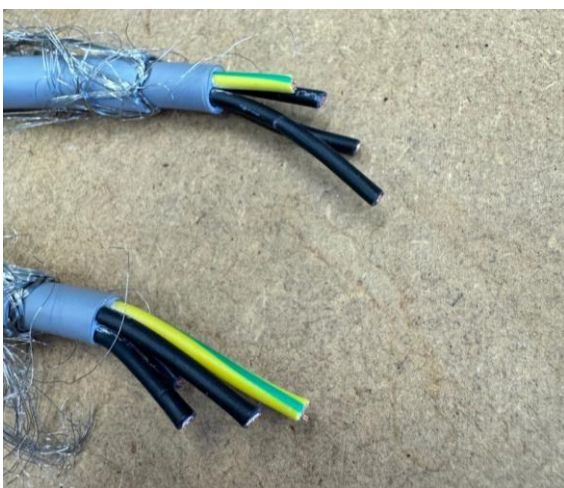
After that, I removed the outer insulation carefully and gently pulled back the silver shielding that protects against magnetic interference.



Next, I stripped the smallest internal wires to expose the conductors.



I then cut each wire so that all of them matched perfectly in length and spread them apart to prevent interference.



After that, I soldered the corresponding wires together and covered each connection with heat-shrink tubing, heating it until it fixed firmly.

Finally, I reconnected the silver shielding, covered it with insulation, and completed the cable preparation.



REPIF - Userstory A4 - Sensor #1 (16Bit Converter and Gas Module) Maksym Bylym

Table of content

Sensor Configuration and Verification	2
Hardware Verification	3
Enabling I ² C Communication	3
I ² C Sensor Detection	3
Sensor Configuration	4
Manual Sensor Test.....	4

Sensor Configuration and Verification

Before performing software-based measurements, the sensor hardware and communication interfaces were verified to ensure reliable operation.

Hardware Verification

Before software testing, the hardware setup was completed and verified.

First, the pin headers were soldered onto the sensor-related modules.

The pins of the additional blue component were soldered and mounted onto the circuit board.

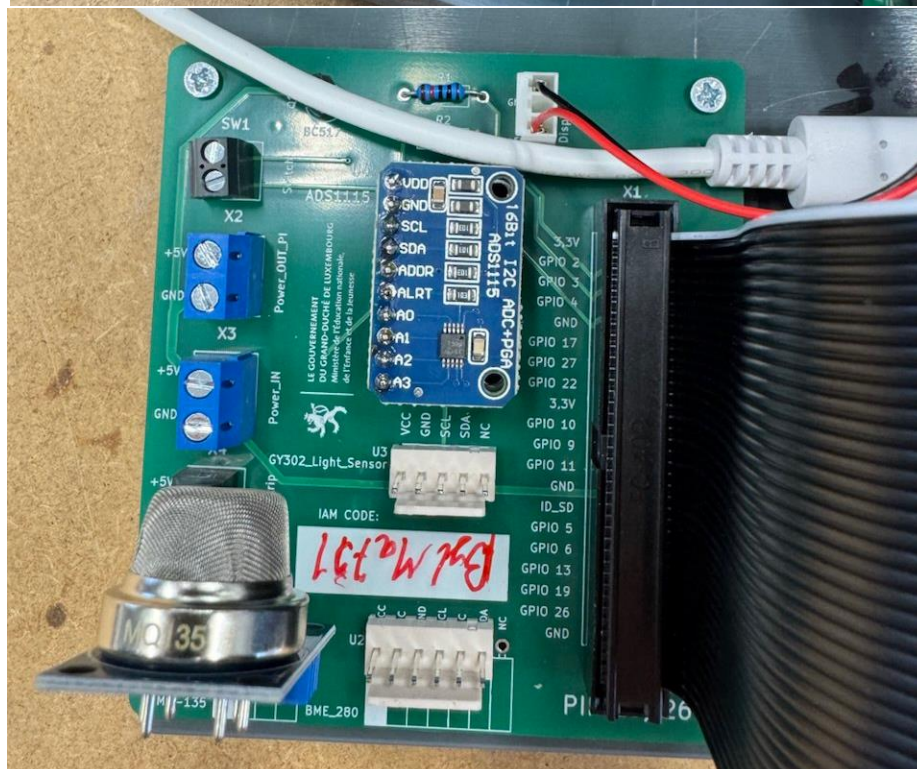
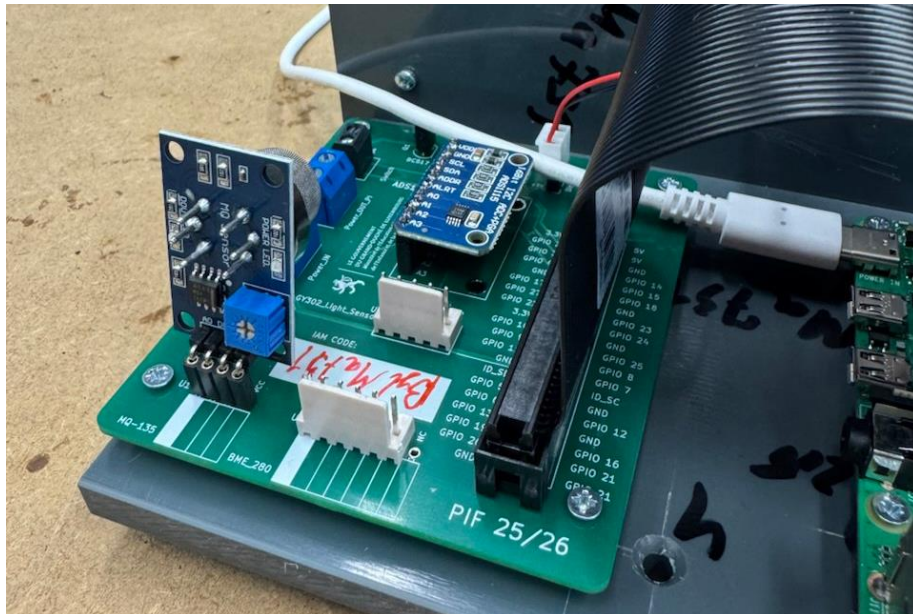
After that, the gas sensor input connector was soldered to the board and the MQ135 gas sensor was inserted into its socket.

All solder joints were visually inspected to ensure mechanical stability and reliable electrical contact.

The Raspberry Pi was powered off and the following checks were performed:

- Sensors were firmly connected with no loose wires
- SDA and SCL lines were correctly connected to the I²C bus
- VCC and GND connections were correctly wired

After verification, the Raspberry Pi was powered back on.



Enabling I²C Communication

To allow communication between the Raspberry Pi and the connected sensors, the I²C interface was enabled using the system configuration tool.

The following command was executed:

```
sudo raspi-config
```

In the menu, the following path was used:

Interface Options → I2C → Enable

After enabling I²C, the Raspberry Pi was rebooted:

```
sudo reboot
```

I²C Sensor Detection

To verify that the sensors were correctly detected on the I²C bus, the I²C tools package was installed.

```
sudo apt install -y i2c-tools
```

The I²C bus was then scanned using:

```
i2cdetect -y
```

The scan displayed the I²C addresses of the connected sensors.

Detected addresses confirmed that the sensors were electrically connected and responding correctly on the I²C bus.

Sensor Configuration

After detecting the sensors, their I²C addresses were added to the configuration file so that the measurement software could access each sensor correctly.

The configuration file was edited:

```
sudo nano /opt/station/config.toml
```

The detected addresses were inserted into the I²C section:

```
[i2c]
bme280 = "0x76"
bh1750 = "0x23"
mq135 = "0x5a"
After saving the file, the configuration was complete.
```

Manual Sensor Test

Before enabling automatic measurements, the sensor measurement script was executed manually to verify correct readings.

```
python3 /opt/station/measure.py
```

The script successfully displayed values such as temperature, humidity, pressure, light intensity, and gas sensor readings.

This confirmed that the hardware setup, configuration, and software were functioning correctly.

REPIF – A5

BMP280 & BH1750 Sensor Installation

DOCUMENTATION

Maksym Bylym

Task Description	3
Materials Used	3
Sensor Installation	3
BMP280 Sensor (HW-611)	3
BH1750 (GY-302) Light Sensor	5
Wire Preparation and Trimming	5
Integration into the REPIF Station	6
Sensor Testing	7
I ² C Address Detection	7
Python Test Scripts	7
Results	7

Task Description

The goal of this task was to connect and test the BMP280 pressure/temperature sensor and the BH1750 light sensor using a custom PCB interfaced with a Raspberry Pi.

The task involved preparing the sensor wiring, soldering the connections, trimming excess wires, insulating cut ends with heat-shrink tubing, and verifying sensor functionality through Python test scripts.

Materials Used

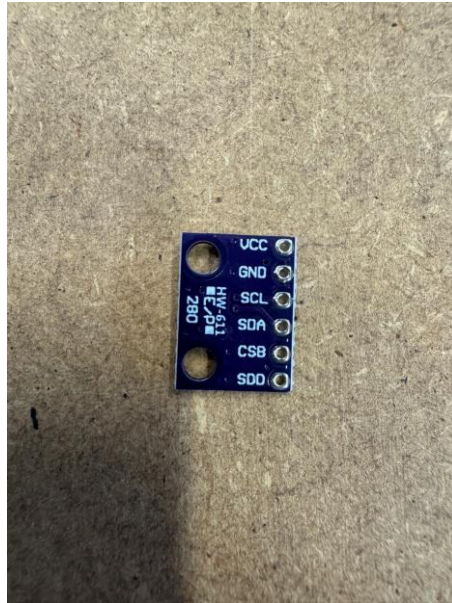
- BMP280 pressure/temperature sensor module (HW-611)
- BH1750 (GY-302) light sensor module
- Raspberry Pi
- Custom PCB (PIF 25/26)
- Wires (black, red, orange/red, green)
- Heat-shrink tubing
- Wire cutters (Knipex pliers)
- Soldering iron and solder
- Cable ties

Sensor Installation

Both sensor modules – the BMP280 and the BH1750 – were provided as separate blue PCB boards. Each module required wiring to be soldered directly into the pin holes.

BMP280 Sensor (HW-611)

The BMP280 sensor module (labeled HW-611 / E/P1 280) features six pins: VCC, GND, SCL, SDA, CSB, and SDD. The module communicates over I²C protocol.

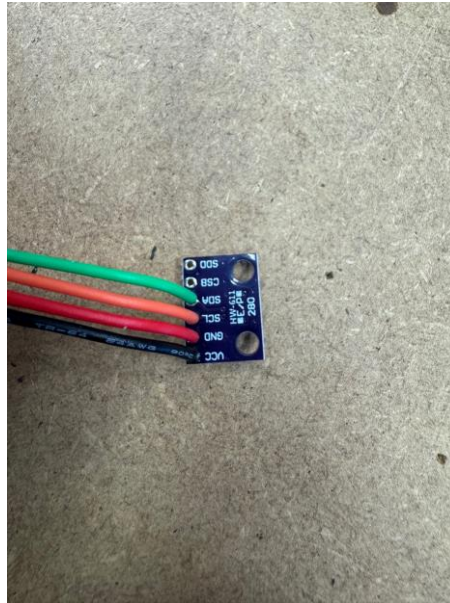


Picture 1. BMP280 (HW-611) sensor module – pin side view



Picture 2. BMP280 (HW-611) sensor module – component side view

Four wires were soldered to the sensor: VCC, GND, SCL, and SDA. The CSB and SDD pins were not used for I²C operation and were left unconnected.



Picture 3. BMP280 sensor with soldered wires (VCC, GND, SCL, SDA)

BH1750 (GY-302) Light Sensor

The BH1750 light sensor (GY-302 module) has five pins: VCC, GND, SCL, SDA, and ADDR. Five wires were prepared for connection to the custom PCB.



Picture 4. GY-302 (BH1750) light sensor with all five wires soldered

Wire Preparation and Trimming

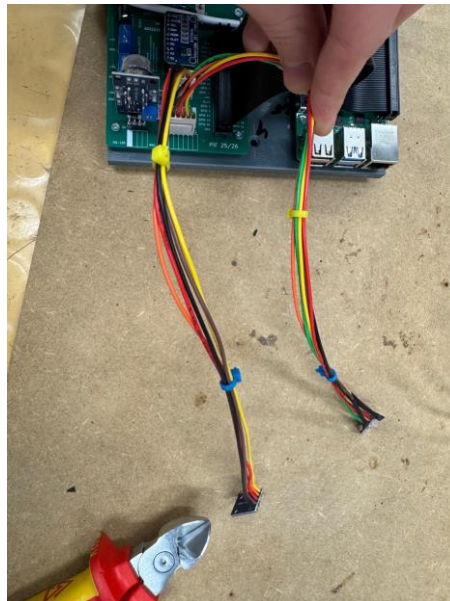
One sensor cable assemblies was with more wires than required for the I²C connections. The excess wires were carefully cut to length and the cut ends were covered with heat-shrink tubing to prevent accidental short circuits and ensure electrical safety.

The heat-shrink tubing was applied over each trimmed wire end and heated until it contracted firmly, providing secure insulation.



Picture 5. Cut wires with heat-shrink insulation applied to the trimmed ends

Once the wiring was prepared, the sensors were connected to the custom PCB. The cable assemblies were routed from the PCB connectors to their respective sensor modules and secured with cable ties to maintain organization and prevent strain on the solder joints.



Picture 6. Both sensor cable assemblies connected to the PCB, secured with cable ties

Integration into the REPIF Station

The assembled PCB with all connected sensors was mounted onto the PVC base plate of the REPIF station. The Raspberry Pi was connected to the PCB via the GPIO ribbon cable. Both the BMP280 and BH1750 sensors were routed and positioned on the station for environmental measurement.



Picture 8. Full station view showing display, PCB, Raspberry Pi, and sensor wiring

Sensor Testing

I²C Address Detection

To verify that both sensors were correctly connected on the I²C bus, the following command was executed:

```
sudo i2cdetect -y 1
```

The scan confirmed that both sensor modules were electrically connected and responding on the I²C bus. The detected addresses were then added to the configuration file of the station software.

Python Test Scripts

After detecting the sensors, Python scripts were executed to read live data from both modules. The BH1750 script returned valid ambient light readings in lux, and the BMP280 script returned valid temperature and atmospheric pressure readings.

This confirmed correct module placement on the PCB, proper routing, successful I²C communication, and functional sensor output.

Results

- BH1750 (light sensor): Returned ambient brightness values in lux
- BMP280 (pressure/temperature sensor): Returned temperature values (°C) and atmospheric pressure (hPa)
- All connections verified electrically safe with heat-shrink insulation on trimmed wires

Both sensors were successfully installed, wired, and verified as fully functional on the REPIF Portable Indoor Feedback Station.

REPIF

Wooden Box Assembly & Sensor Mounting

Maksym Bylym

1TPIFI

Table of Content

1. Introduction	3
2. Materials Used	3
3. Drilling the Hole in the Side Panel	4
4. Assembling the Box	5
5. Mounting the Sensors	6
6. Conclusion	7

1. Introduction

The goal of this sprint was to physically assemble a wooden box enclosure for the REPIF station and to mount two sensors on the back panel. The work included cutting and sanding a circular opening in a side panel for a future lighting button, assembling all wooden panels into a complete box structure, drilling four mounting holes in the back panel, and securing two sensors using bolts.

This documentation covers all steps performed, materials used, and includes photographs taken throughout the process.

2. Materials Used

Wooden Panels

- 2 x side panels with dowel pin alignment holes
- 1 x back panel with 4 pre-drilled sensor mounting holes
- 1 x front panel with circular cutout for future lighting button
- 1 x top panel
- 1 x bottom panel
- Multiple wooden dowel pins for panel alignment and joining

Electronic Components

- 1 x BMP280 temperature and humidity sensor
- 1 x BH1750 ambient light intensity sensor

Fasteners

- 4 x bolts and corresponding nuts
- Wooden dowel pin

Tools

- Drill and drill bits

- Screwdriver



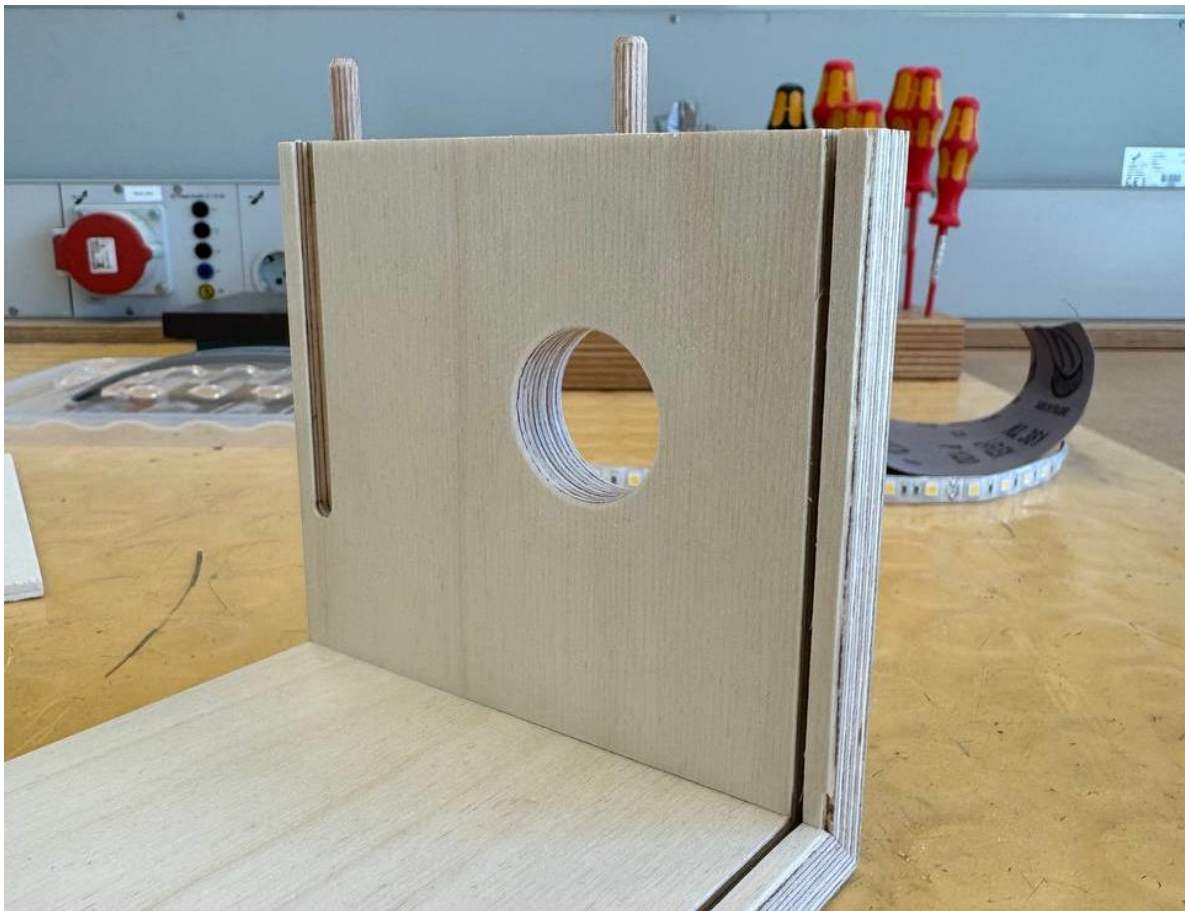
Picture 1. All wooden panels, dowel pins, LED strip, and other components laid out before assembly

3. Drilling the Hole in the Side Panel

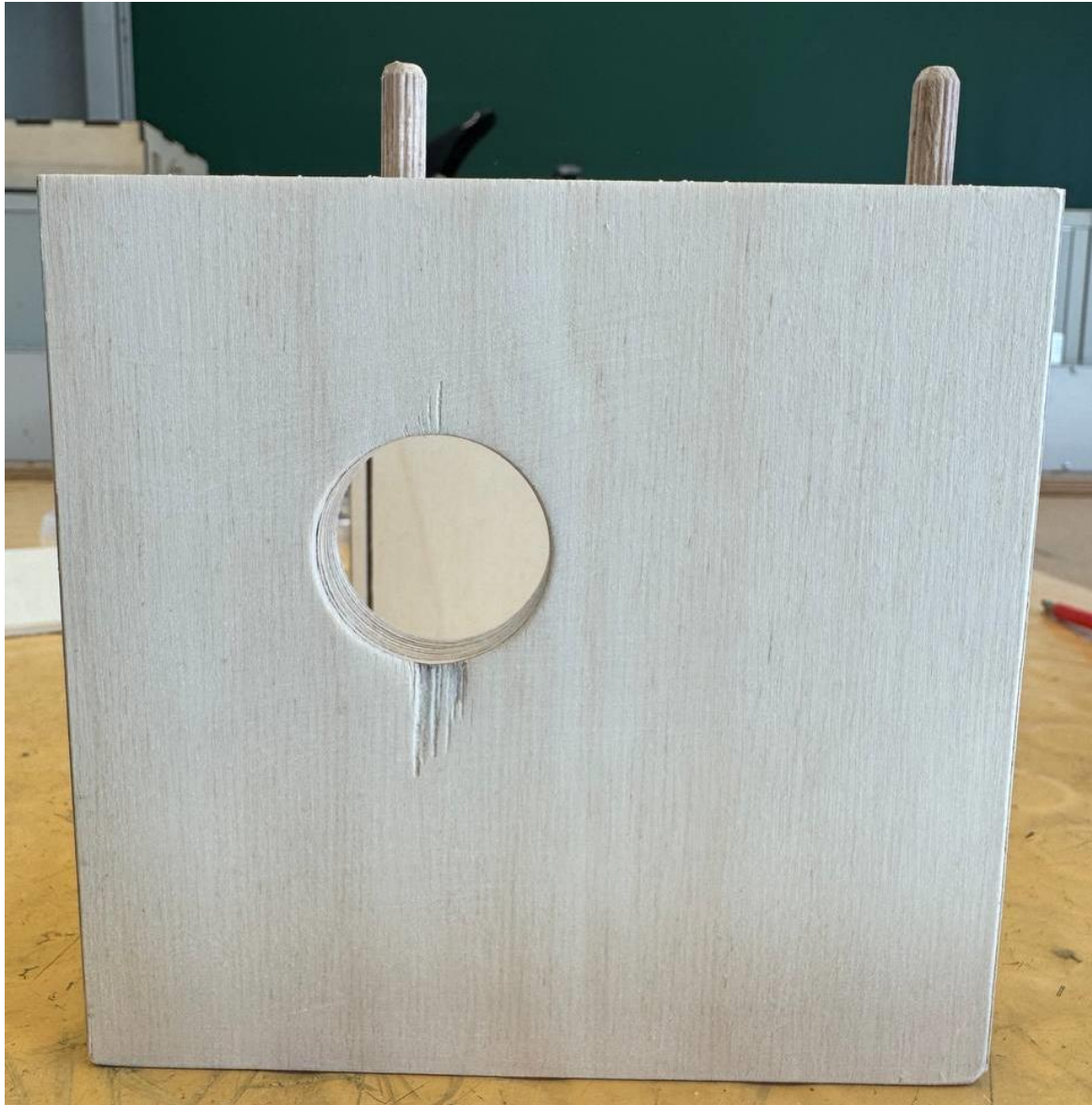
At the start of the sprint, one of the side panels was prepared for a future lighting button. A circular hole was drilled into the panel and then carefully sanded to produce a clean, smooth edge ready for the button installation in the next sprint.

Steps performed:

- A circle was marked on the side panel at the required position.
- A hole was drilled and the circular cutout was completed.
- The edges were sanded with sandpaper until smooth and even.
- The panel was inspected to confirm the opening was clean and correctly shaped.



Picture 2. Front view of the side panel with the circular hole after sanding



Picture 3. Side-angle view of the panel showing the completed circular cutout

4. Assembling the Box

After preparing the side panel, all wooden panels were assembled together to form a complete box enclosure. The panels were joined using wooden dowel pins which were inserted into pre-drilled holes to align and hold the entire structure securely.

Steps performed:

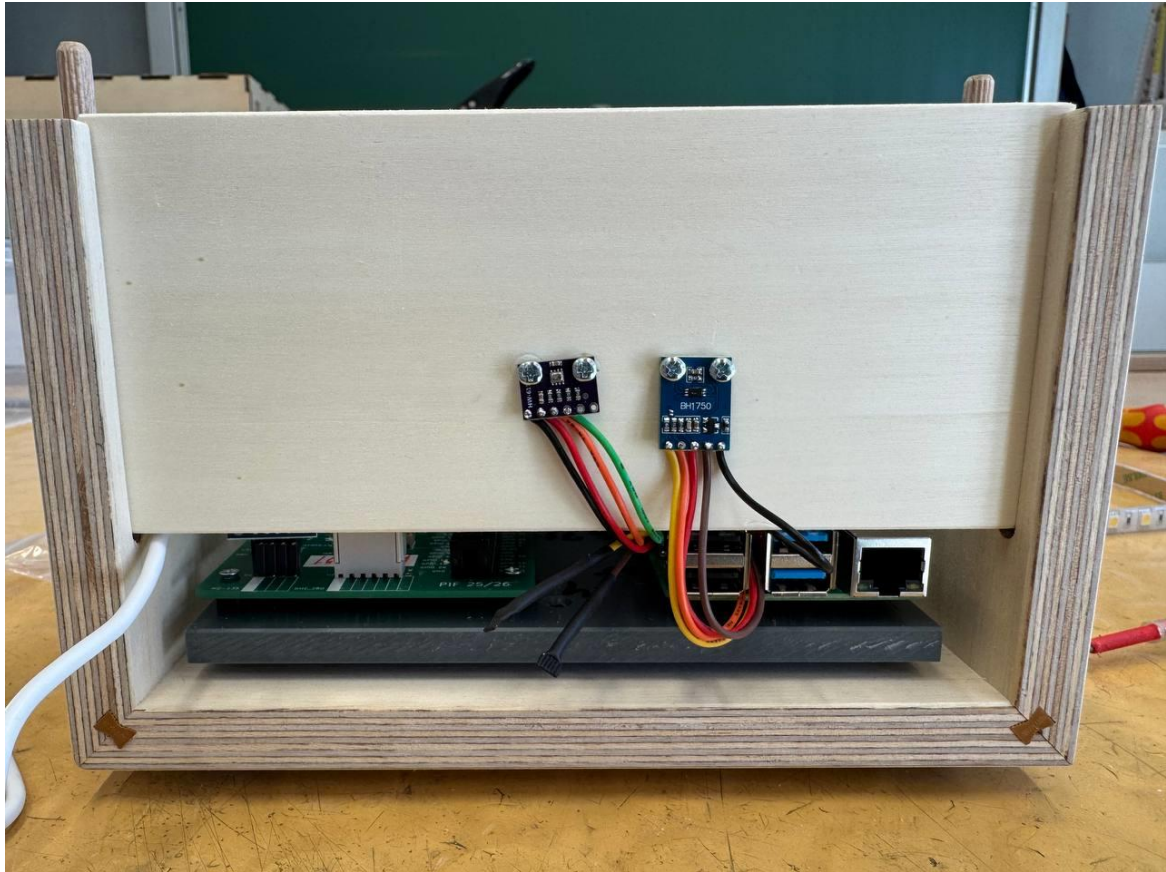
- All panels were identified and oriented correctly.
- Wooden dowel pins were inserted into the alignment holes on each panel.
- The panels were fitted together one by one to form the box.
- The assembled box was checked for proper alignment and stability.

5. Mounting the Sensors

Once the box was assembled, two sensors were mounted onto the back panel. Four small holes had been drilled in the back panel for this purpose. Each sensor was positioned, aligned with the holes, and secured with bolts and nuts. Colour-coded wires were then routed from the sensors down to the Raspberry Pi inside the enclosure.

Steps performed:

- The back panel was positioned with the four mounting holes accessible from the outside.
- The BMP280 temperature and humidity sensor was aligned over two mounting holes and fastened with bolts.
- The BH1750 light intensity sensor was aligned over the remaining two holes and fastened with bolts.
- All connections were checked to ensure they were secure.



Picture 4. Exterior view of the back panel with both sensors (BMP280 and BH1750) mounted using bolts



Picture 5. Interior view of the assembled box showing the Raspberry Pi, wiring, ribbon cable, and sensor connections

6. Conclusion

During this sprint, all planned tasks were completed successfully. A circular hole was cut and sanded into a side panel in preparation for the lighting button to be installed in the next sprint. All wooden panels were assembled into a stable box enclosure using dowel pin joints. Two sensors — the BMP280 (temperature and humidity) and BH1750 (light intensity) — were securely mounted on the back panel using bolts and connected to the Raspberry Pi board via colour-coded wires.

The enclosure is now structurally complete and electronically prepared for the next phase of development.

A7 – Lighting

Maksym Bylym

1. User Story

The task A7 is about adding an LED light strip to the Portable Indoor Feedback Station. The goal is to illuminate the station so it is easier to use in different lighting conditions.

2. Requirements

Must do:

- Cut the LED strip to the correct length.
- Solder the red (+) and black (–) wires to the LED strip at the cut point.
- Fix the LED strip to the bottom edge of the housing.
- Test that the LED strip works correctly.
- Write documentation: work steps, list of materials, photos, video, and notes about problems.

Optional (Could):

- Add a button to turn the LED lighting on and off by hand.

3. List of Materials

- LED strip 5V
- Red wire and black wire
- Heat shrink tubing

4. Work Steps

Step 1 – Check components

Before starting, all needed parts were checked. The LED strip, wires, soldering iron, and the housing were all available and ready to use.

Step 2 – Cut the LED strip

The LED strip was cut to the right length so it fits along the bottom edge of the housing. The cut was made at a marked cut point on the strip.

Step 3 – Solder the wires

The red wire (+) and the black wire (–) were soldered to the connection points at the end of the LED strip. Heat shrink tubing was placed over the solder joint to protect it.

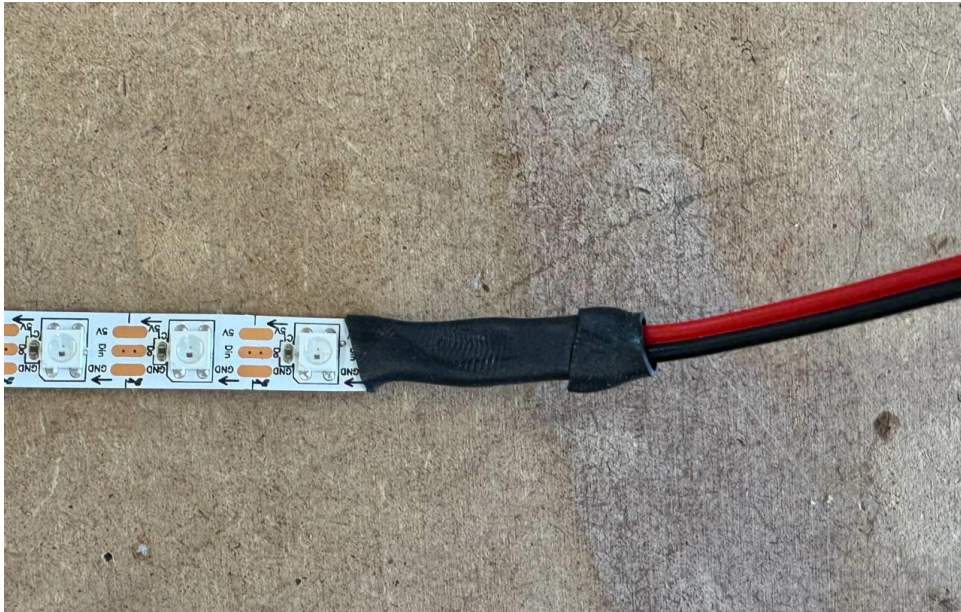


Figure 2 – LED strip with soldered red (+) and black (–) wires, covered with heat shrink tubing

Step 4 – Connect to the PCB board

The wires from the LED strip were connected to the custom PCB board. The board has screw terminal connectors labeled "LED_Strip", "Power_IN", and "Power_OUT_PI". The LED strip was connected to the correct terminal.

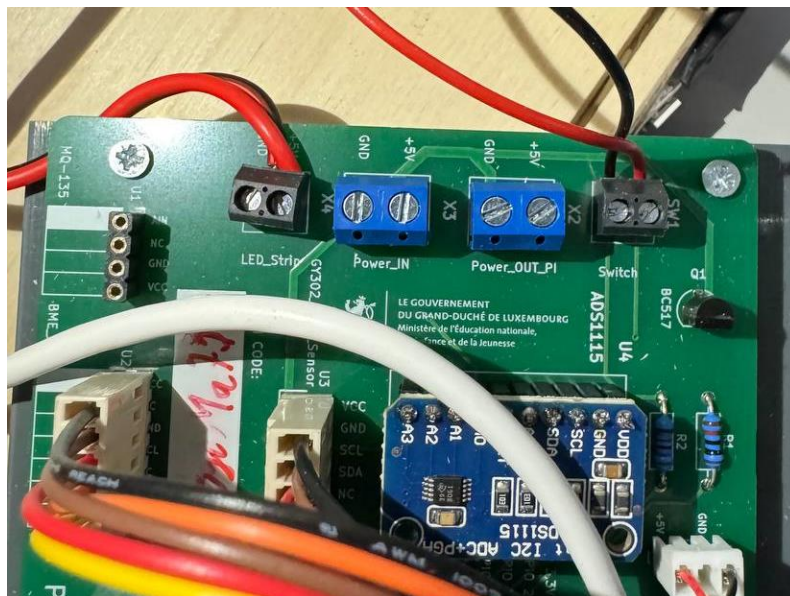


Figure 2 – Custom PCB board showing the ADS1115 module, power terminals, and LED strip connection

Step 5 – Attach the LED strip to the housing

The LED strip was attached to the lower edge of the front side of the housing. The wires were routed inside the box to the PCB board.

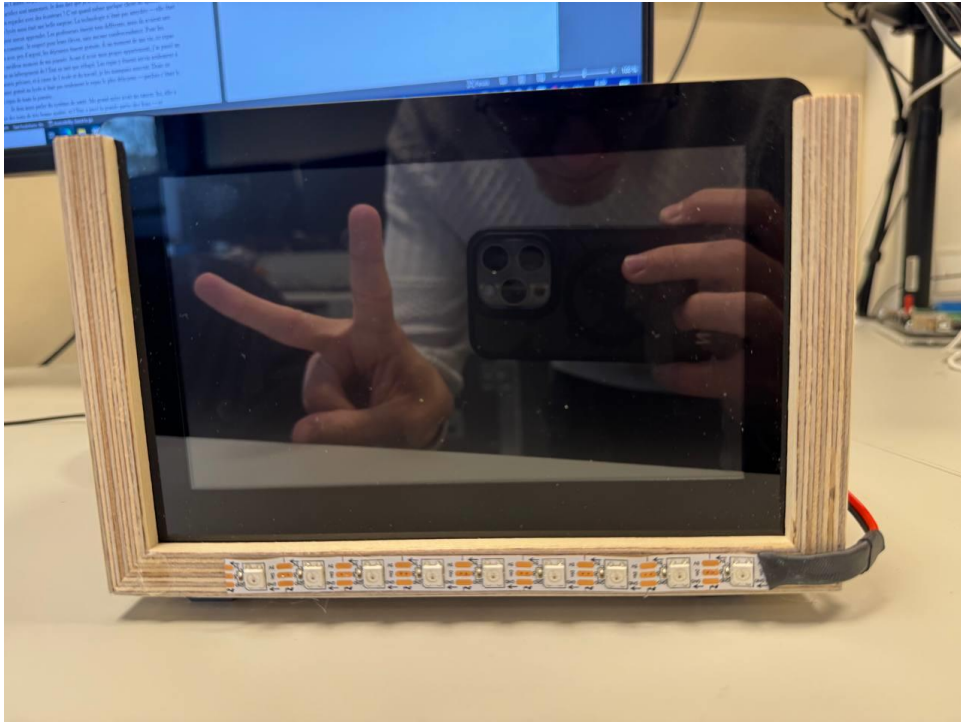


Figure 3 – Front view of the housing showing the display and LED strip mounted at the bottom edge

Step 6 – Button installation

A large blue arcade-style button was added to the back panel of the housing. This button can be used to switch the LED lighting on and off manually. The button is connected through the PCB to a GPIO pin on the Raspberry Pi.

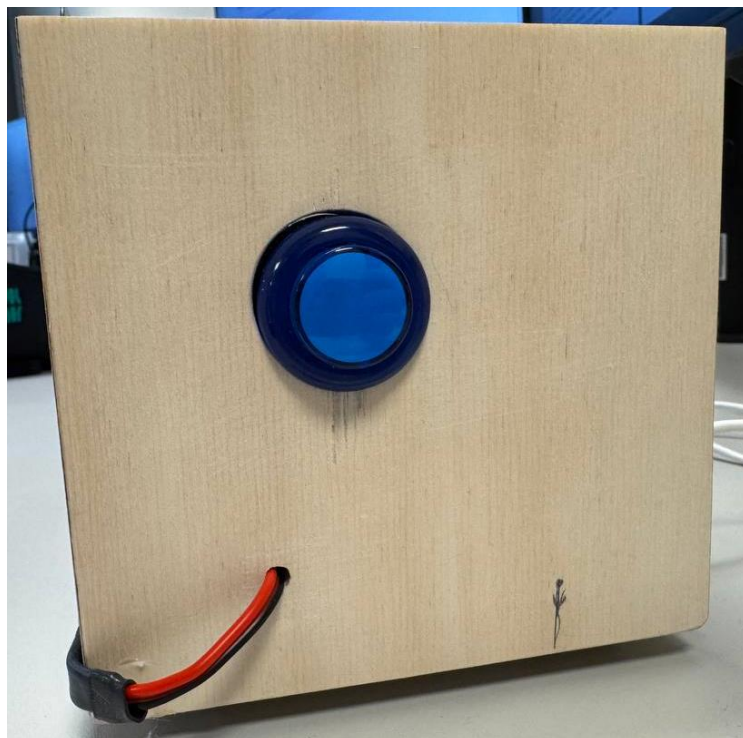


Figure 4 – Back of the housing with the large blue button for manual LED control

Step 7 – Function test

After all connections were made, the station was powered on. The LED strip wasn't working. The button was also tested and didn't work.

5. Conclusion

The LED strip was successfully installed on the station. The lights don't work. Button don't work

REPIF1

Rassberry PI sprint

Maksym Bylym

Installing Raspberry Pi OS	2
Installing the Raspberry Pi Imager	3
OS Customisation (Advanced Settings)	3
SSH Conection	5
RealVNC Viewer	6
Enabling VNC via SSH.....	6
Physical configurations	8
1. Mounting the Raspberry Pi on the Display Frame	8
2. Connecting the Display and Peripherals	9
3. Configuring Screen Orientation	10
Installing Docker	11
1. Update the system.....	11
2. Download and install Docker	11
3. Add the current user to the Docker group	11
4. Create the Docker container	11
Unzipping the Project Files	12
1. Navigate to the Downloads folder	12
2. Unzip the resource archive	12
3. Set correct ownership for the web directory	12
4. Copy firmware and scripts into the station directory	12
5. Copy the configuration file.....	12
Configure autostart	13
1. Install required packages	13
2. Create the temporary data directory.....	13
3. Start the measurement script	13
4. Start the Docker container	13
5. Wait for the server to initialize	13
6. Launch Chromium in kiosk mode.....	13
7. Reboot the Raspberry Pi.....	14
Configuration File (config.toml)	14
Sensor Output Example	14

Installing Raspberry Pi OS

Installing the Raspberry Pi Imager

1. Download and install **Raspberry Pi Imager** from the official website.
2. Launch the application and choose:
 - **Device:** Raspberry Pi 4
 - **Operating System:** Raspberry Pi OS (64-bit)
 - **Storage:** Your SD card
3. Click **Next** to continue.

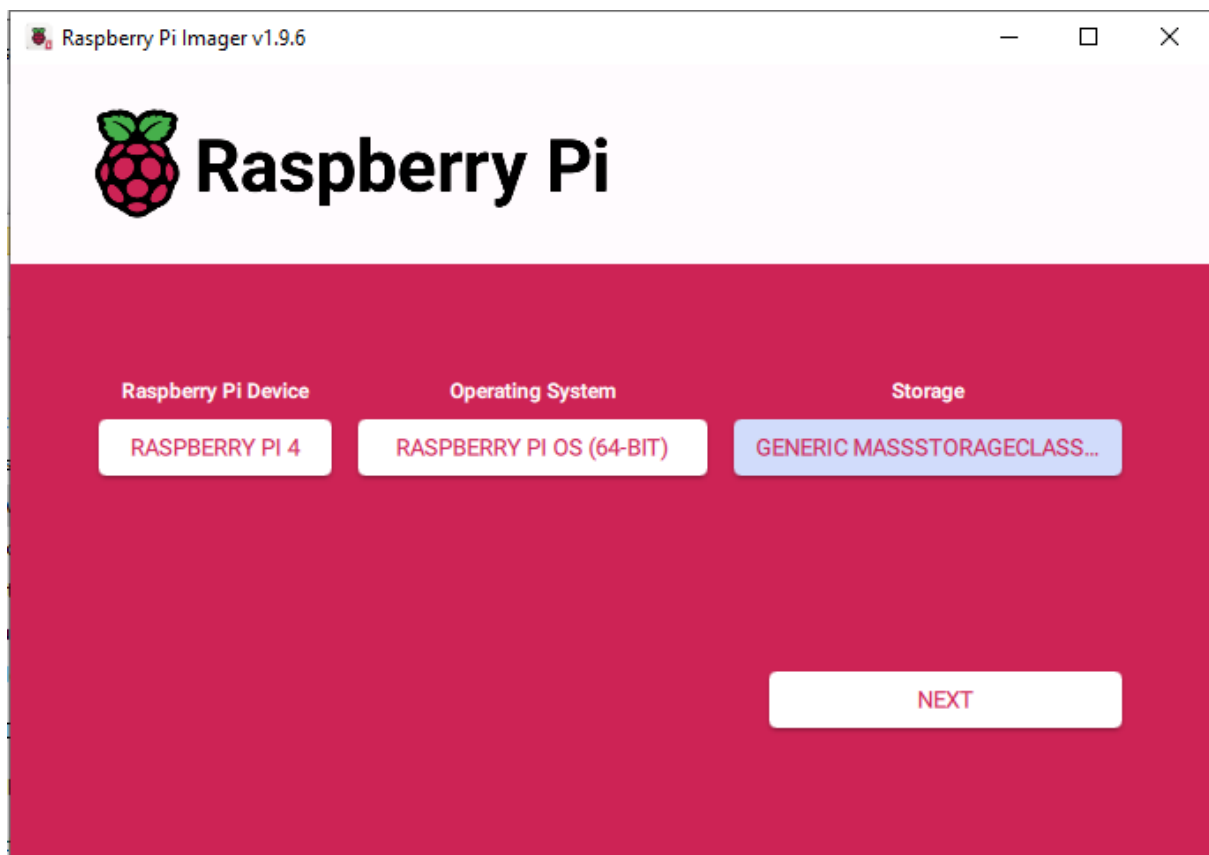


Figure 1. Device, OS and Storage selection in Raspberry Pi Imager

OS Customisation (Advanced Settings)

General settings:

- **Hostname:** bylma731
- **Username:** bylma731
- **Password:** *****
- **Wireless LAN:**
 - SSID: LPEM_B13
 - Password: *****
 - Country: LU

- **Locale:**
 - Time zone: *Europe/Luxembourg*
 - Keyboard layout: *gb*

The image shows a window titled "OS Customisation" with three tabs: "General", "Services", and "Options". The "General" tab is active. It contains several configuration options, each with a checked checkbox:

- Set hostname:** A text box containing "bylma731" followed by ".local".
- Set username and password:** A text box for "Username" containing "bylma731" and a password box with 10 dots.
- Configure wireless LAN:** A text box for "SSID" containing "LPEM_B13" and a password box with 10 dots. Below these is an unchecked checkbox for "Hidden SSID".
- Wireless LAN country:** A dropdown menu showing "LU".
- Set locale settings:** A text box for "Time zone" showing "Europe/Luxembourg" and a dropdown menu for "Keyboard layout" showing "gb".

At the bottom of the window are two buttons: "CANCEL" and "SAVE".

Figure 2. General system configuration (hostname, Wi-Fi, locale)

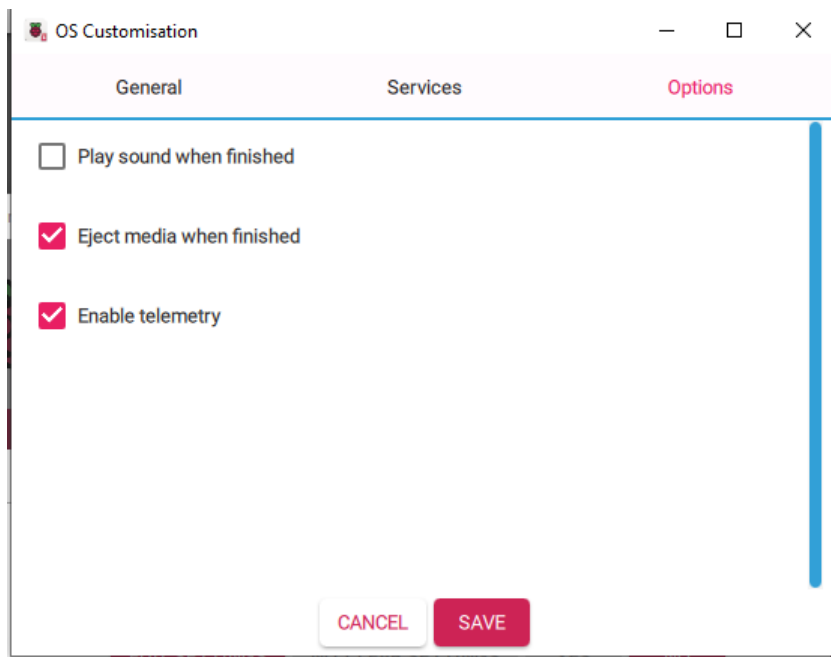


Figure 3. Additional OS options

SSH Conection

Initially, I enabled SSH access to connect to the Raspberry Pi remotely.

Later, I switched to **RealVNC Viewer**, as it provides a graphical interface and is more convenient for daily use.

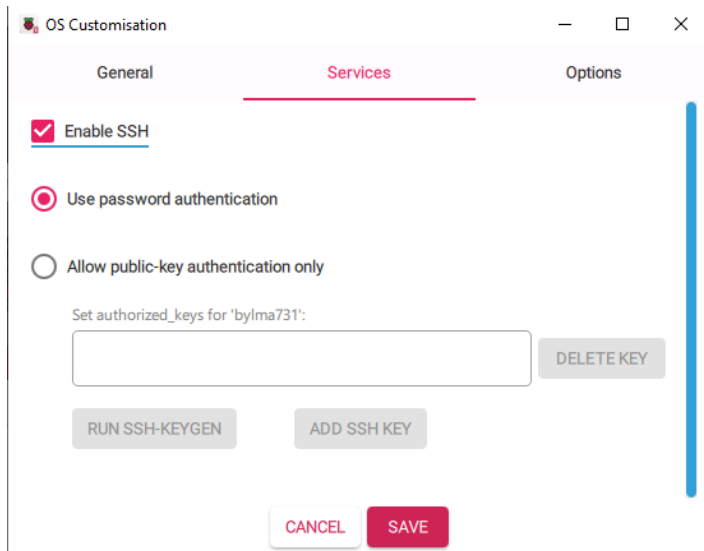
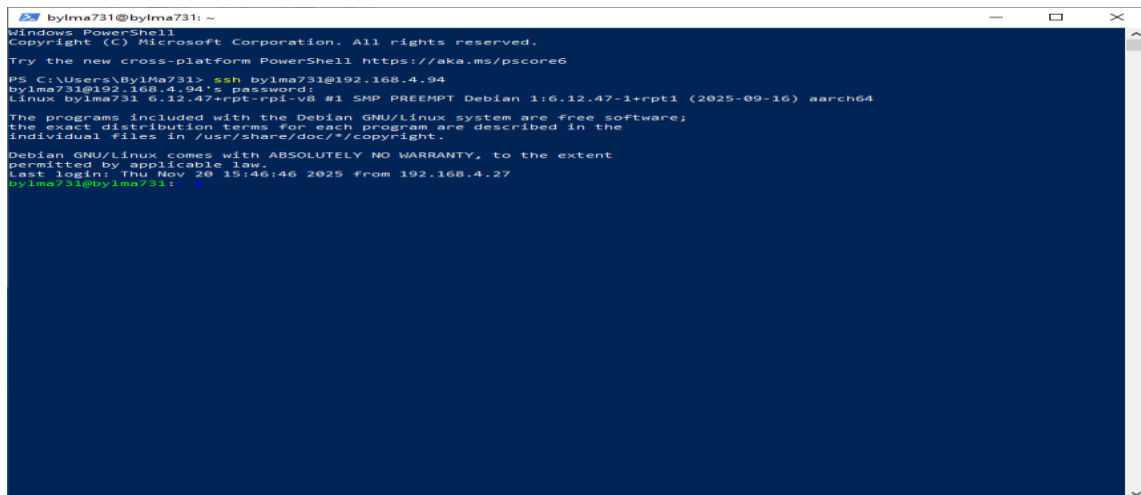


Figure 4. Enabling SSH in Raspberry Pi Imager



RealVNC Viewer

To enable graphical remote access, I installed **RealVNC Viewer** on my computer.

Before using it, VNC support must be activated on the Raspberry Pi.

Enabling VNC via SSH

- Connect to the Raspberry Pi using SSH.
- Open the Raspberry Pi configuration tool:

```
sudo raspi-config
```

- Navigate to: **Interface Options** → VNC

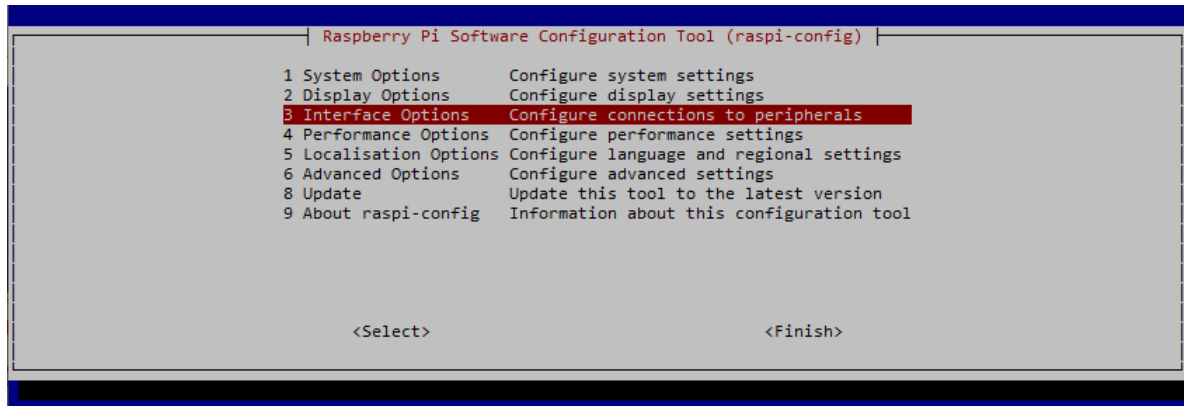
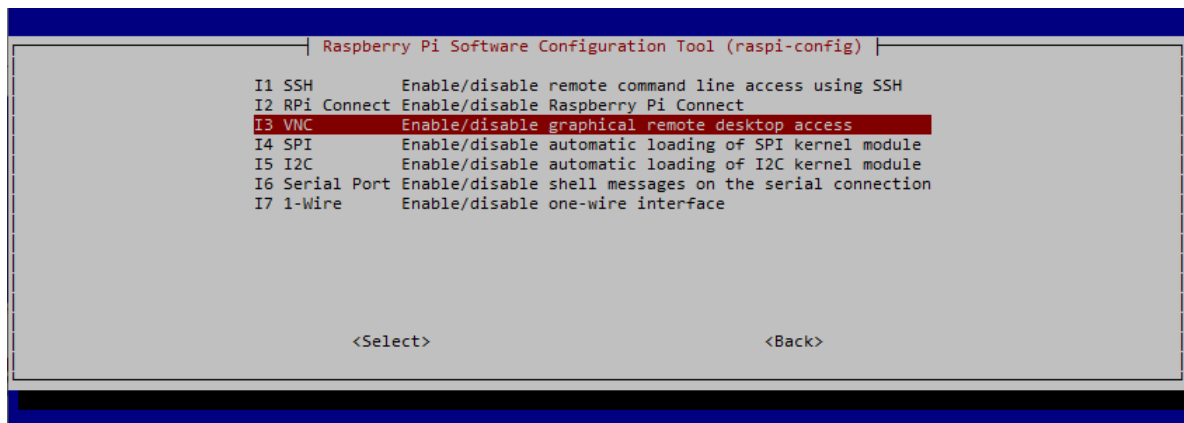


Figure 5. Raspberry Pi Configuration Tool – Interface Options



- Select VNC and enable the VNC Server.

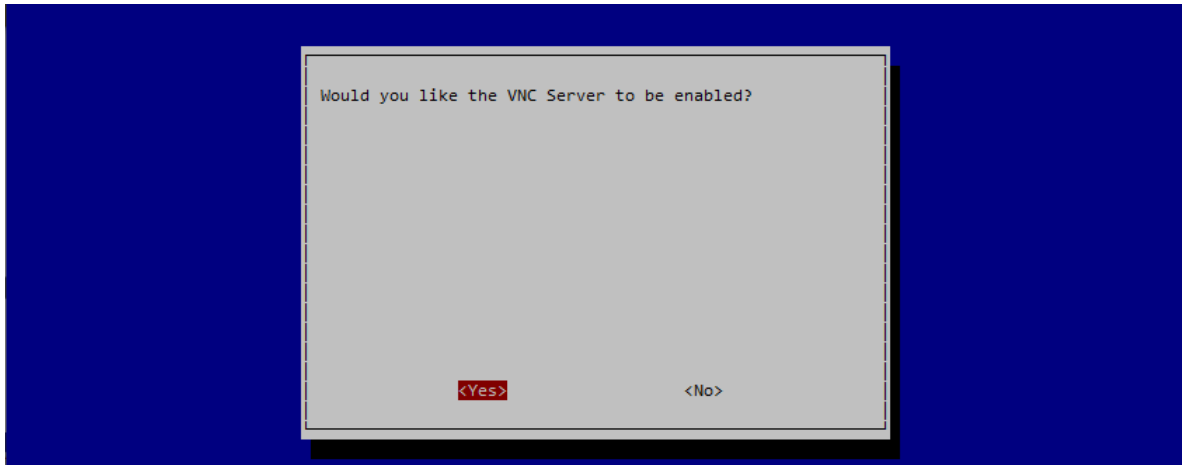


Figure 6. Enabling the VNC Server

Physical configurations

In this step, the Raspberry Pi, touchscreen display, and sensors were physically assembled. Proper hardware installation ensures that the station operates reliably during measurements.

1. Mounting the Raspberry Pi on the Display Frame

The Raspberry Pi was attached to the back of the touchscreen using the provided mounting brackets. This secures the board and prevents movement during operation.

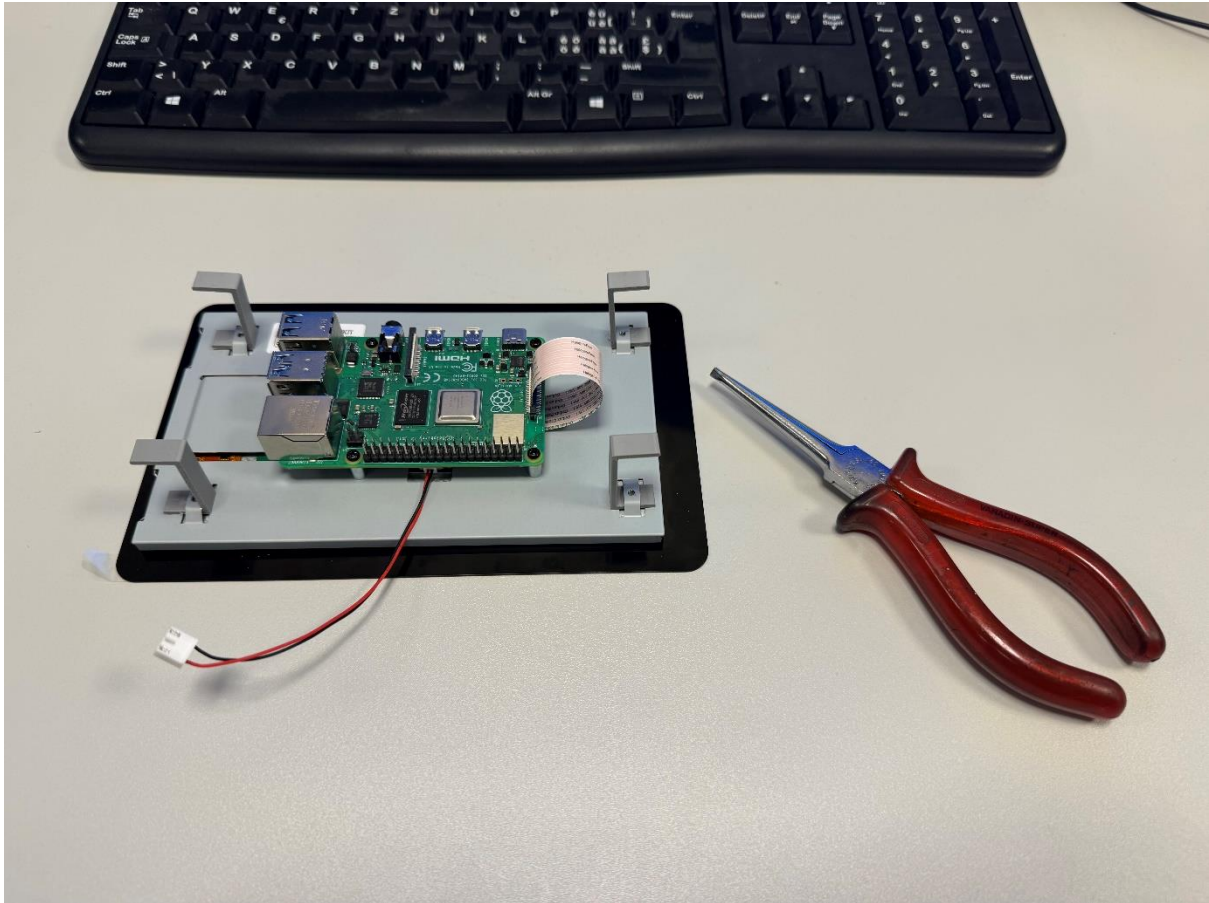


Figure 7. Raspberry Pi placed on the touchscreen frame

2. Connecting the Display and Peripherals

After mounting the Raspberry Pi, the display ribbon cable and power cable were connected. Additional peripherals, such as keyboard, mouse, and network connection, were attached for configuration.



Figure 8. Display connected and Raspberry Pi powered during initial setup

3. Configuring Screen Orientation

Once the touchscreen was connected, the screen orientation was adjusted to match the installation direction.

Navigation:

Menu → Preferences → Screen Configuration → Orientation

The display was rotated to the correct orientation for proper viewing.

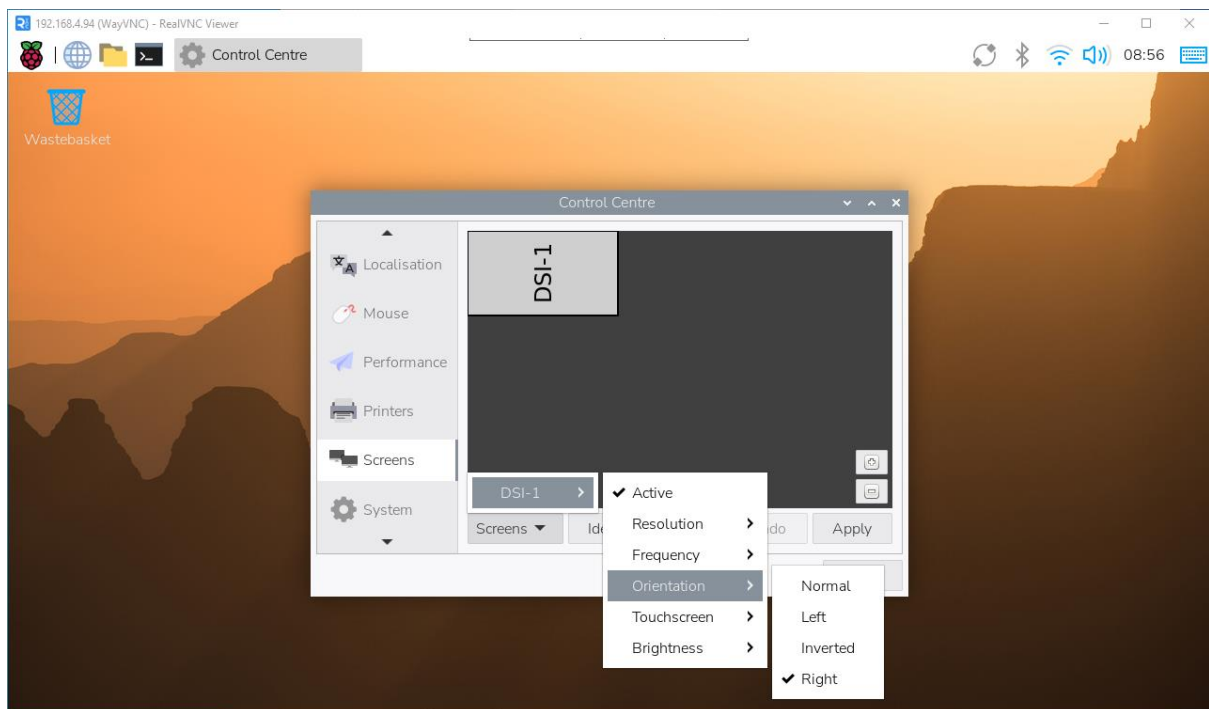


Figure 10. Adjusting the display orientation via Screen Configuration

Installing Docker

Docker was installed on the Raspberry Pi to run the PHP–Apache environment required for the measurement station.

The following commands were executed:

1. Update the system

```
sudo apt update && sudo apt upgrade -y
```

2. Download and install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
sudo sh get-docker.sh
```

3. Add the current user to the Docker group

This allows running Docker without using sudo.

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

4. Create the Docker container

A PHP 8.4 Apache server container is created with mounted folders for data and web files.

```
docker run -d --name php --restart unless-stopped -p 8080:80 -v /tmp/station:/tmp/station -v /opt/station/www:/var/www/html php:8.4-apache
```

Unzipping the Project Files

After installing Docker, the project resources were downloaded from Microsoft Teams using Chromium on the Raspberry Pi.

These files contain the firmware, scripts, and web interface required for the measurement station.

1. Navigate to the Downloads folder

```
cd ~/Downloads
```

2. Unzip the resource archive

This extracts the content into a folder named **ressourcen**.

```
unzip Ressourcen.zip -d ressourcen
```

3. Set correct ownership for the web directory

The web interface must be readable by the web server:

```
sudo chown -R root:root /opt/station/www  
sudo chmod -R 755 /opt/station/www
```

4. Copy firmware and scripts into the station directory

```
sudo cp -r ~/Downloads/ressourcen/Ressourcen/Firmware\ und\ Scripts/* /opt/station/
```

5. Copy the configuration file

```
sudo cp ~/Downloads/ressourcen/Ressourcen/Firmware\ und\ Scripts/config.toml  
/var/station/www
```

Configure autostart

To ensure the measurement station starts automatically after the Raspberry Pi boots, several commands were executed. These prepare the environment, start the measurement script, and launch Chromium in kiosk mode.

1. Install required packages

wtype is needed for simulated keystrokes in kiosk mode.

```
sudo apt -y install wtype
```

2. Create the temporary data directory

This folder stores measurement results.

```
mkdir -p /tmp/station
```

3. Start the measurement script

The script runs in the background and writes sensor values to result.csv.

```
python3 /opt/station/measure.py > /tmp/station/result.csv &
```

4. Start the Docker container

This launches the PHP–Apache web server used to display the results.

```
docker start php
```

5. Wait for the server to initialize

```
sleep 5
```

6. Launch Chromium in kiosk mode

This opens the dashboard at boot, full-screen, without menus or pop-ups.

```
chromium http://localhost:8080 --kiosk --noerrdialogs --disable-infobars --no-first-run --password-store=basic --enable-features=OverlayScrollbar --start-maximized &
```

7. Reboot the Raspberry Pi

```
sudo reboot
```

Configuration File (config.toml)

The following configuration file defines the station's behavior, including sampling interval, sensor interfaces, and destination server.

```
station_serial = "STATION01"    # Serial number of the station
interval = 2                    # Time between measurements (seconds)
toggle_intensity = 100          # Light level at which the LED should turn on

[i2c]                          # I2C addresses of the sensors (empty => no-sensor-mode)
bme280 = ""
bh1750 = ""
mq135 = ""

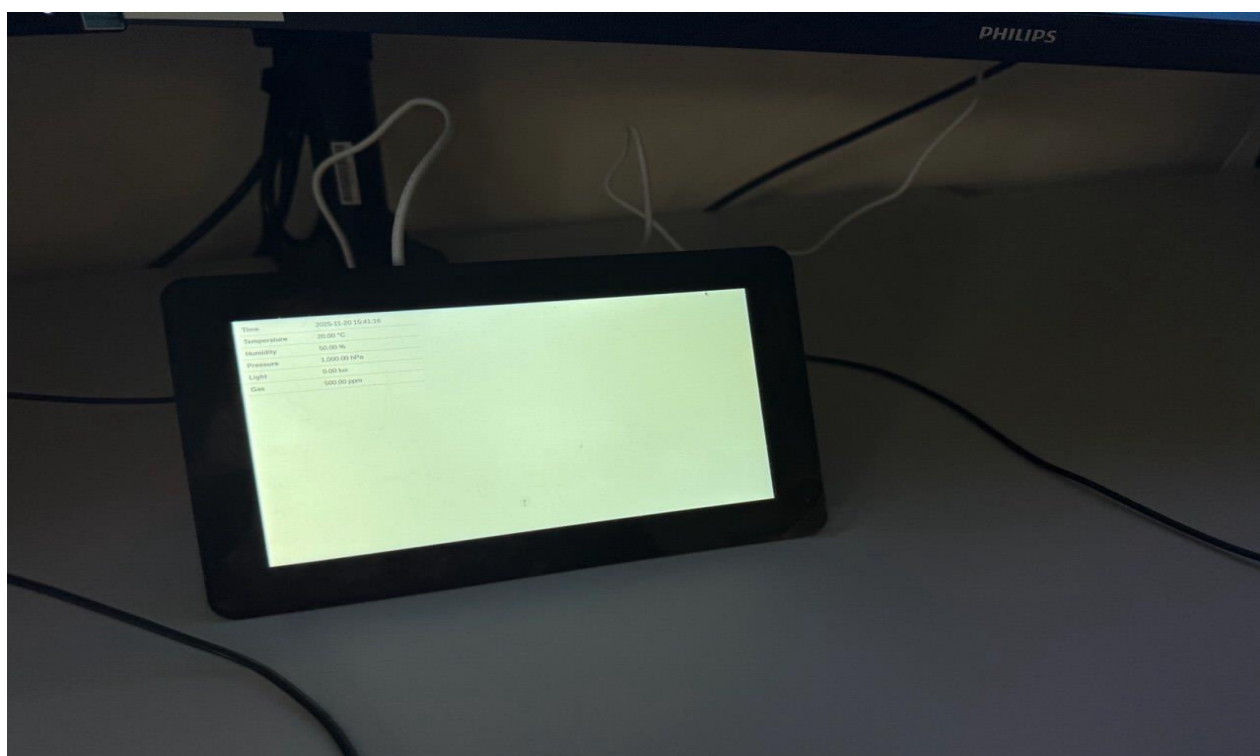
[destination]                  # Information about the server
ip = "127.0.0.1"               # IP address or hostname (dummy is OK for now)
path = "/"                      # Path to script that accepts measurements
```

Sensor Output Example

The following screenshot shows an example of sensor readings displayed through the station's web interface.

192.168.4.94 (WayVNC) - RealVNC Viewer	
Time	2025-11-20 15:34:40
Temperature	20.00 °C
Humidity	50.00 %
Pressure	1,000.00 hPa
Light	0.00 lux
Gas	500.00 ppm

Figure 11. Example of live sensor data displayed in the dashboard



REPIF1

DataBase sprint

Maksym Bylym

Table of Content

MCD Model	3
MLD Model	4
1. Users.....	5
2. Station	5
3. Measurements	5
4. Collections.....	6
5. Collection_Measurement (N:M)	6
6. Friendship (User $\leftrightarrow$ User)	7
7. SHARED_COLLECTION (User $\leftrightarrow$ Collection)	7

MCD Model

The CDM (Conceptual Data Model) shows entities, attributes, and relationships.

Entities:

- Users
- Stations
- Measurements
- Collections
- Friendships (self-relationship)
- Shared Collections
- Collection Measurement

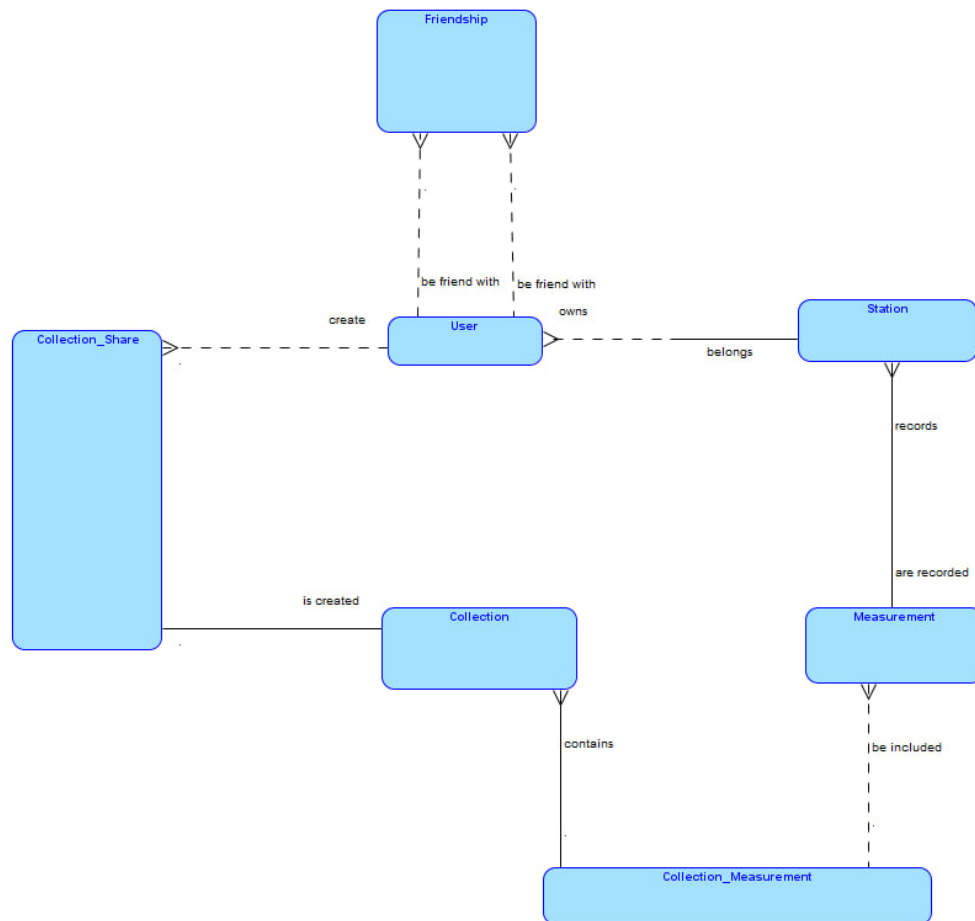


Figure 1. The Conceptual Data Model

MLD Model

The MLD (Logical Data Model) with primary keys, foreign keys, and data types.

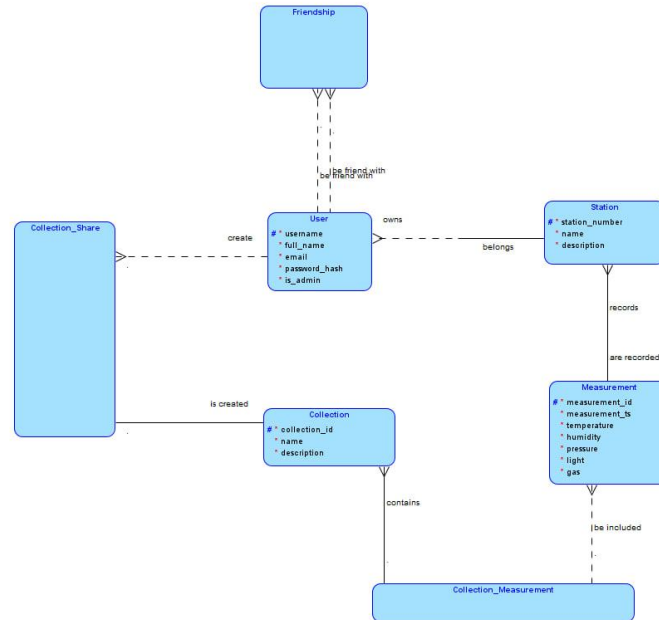


Figure 1. The Logical Data Model

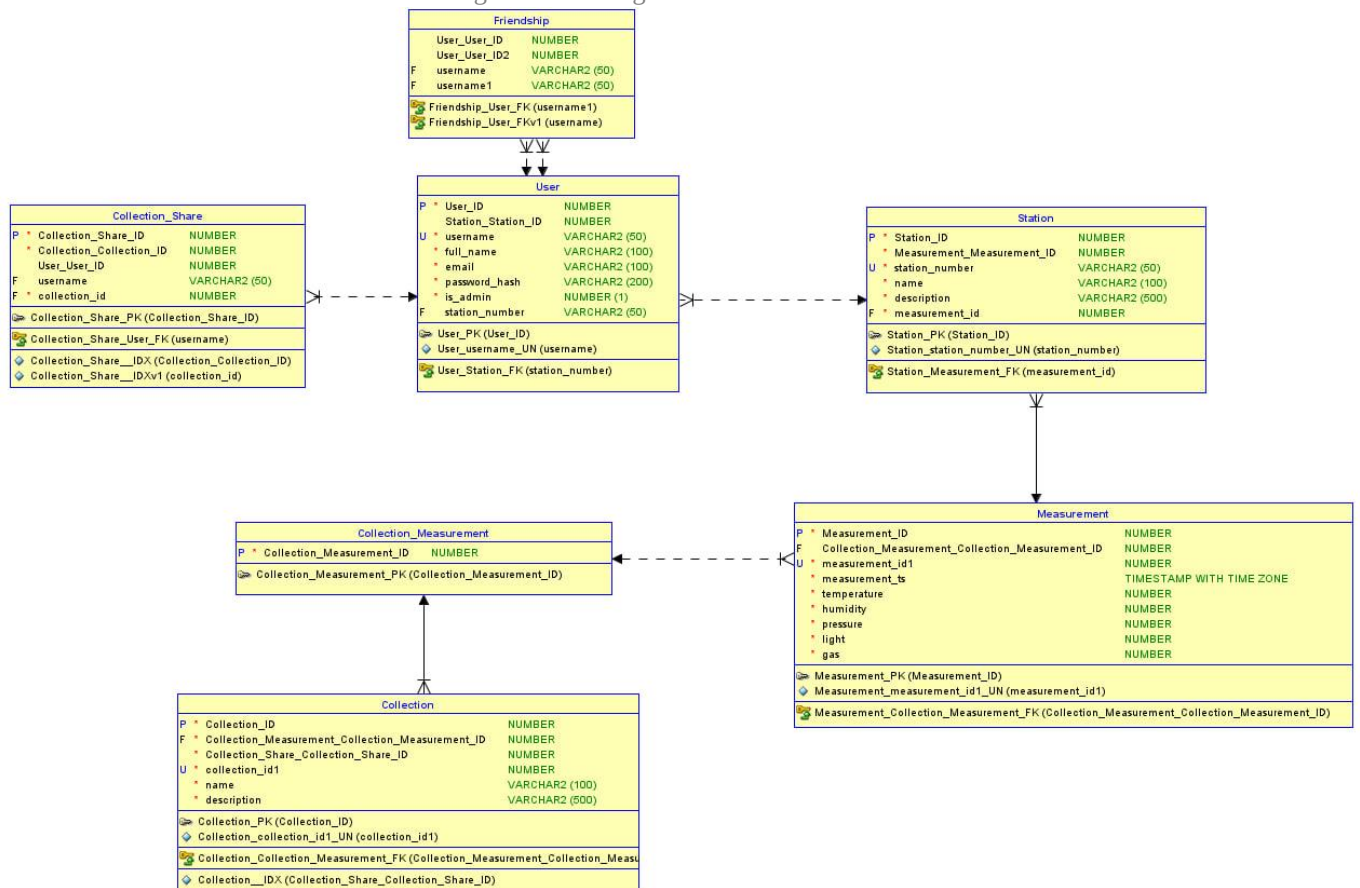


Figure 1. The Logical Data Model (engineer mode)

SQL Commands

1. Users

```
CREATE TABLE USERS (  
  
    username    VARCHAR2(50) CONSTRAINT pk_users PRIMARY KEY,  
  
    full_name   VARCHAR2(100) NOT NULL,  
  
    email       VARCHAR2(100) NOT NULL,  
  
    password_hash VARCHAR2(200) NOT NULL,  
  
    is_admin    NUMBER(1)   DEFAULT 0  
  
);
```

2. Station

```
CREATE TABLE STATIONS (  
  
    serial_number VARCHAR2(50) CONSTRAINT pk_stations PRIMARY KEY,  
  
    owner_username VARCHAR2(50) NOT NULL,  
  
    name           VARCHAR2(100) NOT NULL,  
  
    description    VARCHAR2(500),  
  
    CONSTRAINT fk_stations_owner  
  
        FOREIGN KEY (owner_username)  
  
        REFERENCES USERS(username)  
  
);
```

3. Measurements

```
CREATE TABLE MEASUREMENTS (  
  
    measurement_id NUMBER  
  
        GENERATED BY DEFAULT AS IDENTITY  
  
        CONSTRAINT pk_measurements PRIMARY KEY,  
  
    station_serial VARCHAR2(50) NOT NULL,
```

```

measurement_ts TIMESTAMP NOT NULL,

temperature NUMBER,

humidity NUMBER,

pressure NUMBER,

light NUMBER,

gas NUMBER,

CONSTRAINT fk_meas_station

    FOREIGN KEY (station_serial)

    REFERENCES STATIONS(serial_number)

);

```

4. Collections

```

CREATE TABLE COLLECTIONS (

    collection_id NUMBER

        GENERATED BY DEFAULT AS IDENTITY

        CONSTRAINT pk_collections PRIMARY KEY,

    creator_username VARCHAR2(50) NOT NULL,

    name VARCHAR2(100) NOT NULL,

    description VARCHAR2(500),

    CONSTRAINT fk_coll_creator

        FOREIGN KEY (creator_username)

        REFERENCES USERS(username)

);

```

5. Collection_Measurement (N:M)

```

CREATE TABLE COLLECTION_MEASUREMENT (

    collection_id NUMBER NOT NULL,

    measurement_id NUMBER NOT NULL,

```

```

CONSTRAINT pk_coll_meas

    PRIMARY KEY (collection_id, measurement_id),

CONSTRAINT fk_cm_collection

    FOREIGN KEY (collection_id)

    REFERENCES COLLECTIONS(collection_id),

CONSTRAINT fk_cm_measurement

    FOREIGN KEY (measurement_id)

    REFERENCES MEASUREMENTS(measurement_id)

);

```

6. Friendship (User ↔ User)

```

CREATE TABLE FRIENDSHIP (

    user1_username VARCHAR2(50) NOT NULL,

    user2_username VARCHAR2(50) NOT NULL,

    CONSTRAINT pk_friendship

        PRIMARY KEY (user1_username, user2_username),

    CONSTRAINT fk_friend_user1

        FOREIGN KEY (user1_username)

        REFERENCES USERS(username),

    CONSTRAINT fk_friend_user2

        FOREIGN KEY (user2_username)

        REFERENCES USERS(username)

);

```

7. SHARED_COLLECTION (User ↔ Collection)

```

CREATE TABLE SHARED_COLLECTION (

    collection_id NUMBER    NOT NULL,

    username    VARCHAR2(50) NOT NULL,

```



```
CONSTRAINT pk_shared_collection  
  
    PRIMARY KEY (collection_id, username),  
  
CONSTRAINT fk_sc_collection  
  
    FOREIGN KEY (collection_id)  
  
    REFERENCES COLLECTIONS(collection_id),  
  
CONSTRAINT fk_sc_user  
  
    FOREIGN KEY (username)  
  
    REFERENCES USERS(username)  
  
);
```

AUser Story	L3 - Website	to	KW 05
Informal description	As a user or administrator of a station		
	... I want a functional and user-friendly website		
	... so that I can manage and share stations and measured values.		

**Requirements
according to the
work order**

Required

- Documentation: ○ Annotated source code of the website ○ Planning this user story

Without login:

- Login with username and password is possible.
- Registration/creation of a new user account is possible.

Logged-in users:

- Logout is possible.
- A simple welcome page is displayed.
- Your own stations can be listed. The name and description of the stations can be changed.
- Stations not yet assigned to a user can be registered by serial number.
- Your own user account (username, password, email, first and last name) can be edited.
- Measurement data from your own stations can be viewed and filtered by date/time.
- Users can be added as friends. Friends can be displayed and friendships can be ended.
- Measurement data can be added to collections. Users can only select from their own stations. A collection contains measurement data from a specific station from a start date/time to an end date/time. Your own collections can be renamed and deleted.
- Collections can be shared with friends. Sharing can be undone.
- Collections shared with the user can be viewed.

Admins (additional):

- All user accounts can be edited and deleted. Admin rights can be assigned to users. New user accounts can be created.
- All stations can be edited and deleted. New stations can be created.
- Measurement data from all stations can be viewed, filtered by date/time, and deleted.
- Measurement data from all stations can be added to collections. All collections can be renamed and deleted. Admins can only share collections they have created themselves.

Should

- The welcome page contains a dashboard that displays current measurement data from your own stations.
- The website has an appealing and user-friendly design.
- User entries are validated.
- Users cannot be added as friends directly, but via a friend request system that requires confirmation of the request.
- When a friendship is ended, all collections shared between the two users are automatically unshared.

Could

- Registration of stations via QR code.
- Measurement data is displayed using diagrams.
- The website automatically adapts to different screen sizes (responsive).
- Different themes (e.g., light/dark mode). The selected theme is retained even after logging out.
- The website is available in different languages and the selected language remains unchanged after logging out.
- Users can be notified of various events (friend requests, sharing of measurement data, errors, etc.) via email and/or notifications.
- Users can chat with each other on the website.
- Users can send invite links to invite new users to the platform.

Tasks according to the work assignment

1. Authentication and User Management

- Implement user registration with validation.
- Implement login using username and password.
- Implement logout functionality.
- Create a profile page for editing user data (username, password, email, first name, last name).
- Implement validation for all user input fields.
- Implement admin and normal user types

2. Station Management

- Implement a page to list the user's own stations.
- Implement editing of station name and description.
- Implement registration of unassigned stations using a serial number.

3. Measurement Data

- Implement viewing of measurement data for the user's own stations.
- Implement filtering of measurement data by date and time.

4. Friends System

- Implement adding users as friends.
- Implement displaying the friend list.
- Implement removing friends.
- Implement a friend request system requiring confirmation.
- Automatically unshare collections when a friendship ends.

5. Collections

- Implement creation of collections containing measurement data.
- Implement adding measurement data to a collection (only from the user's own stations).
- Implement renaming and deleting collections.
- Implement sharing collections with friends and undoing sharing.
- Implement viewing collections shared with the user.
- Implement automatic data inclusion for collections, allowing new measurement data within the defined time range to be added automatically.

6. Admin Panel

- Implement an admin interface for managing all user accounts.
- Allow admins to edit and delete any user account.
- Allow admins to assign or remove admin rights.
- Allow admins to create new user accounts.
- Implement admin functionality for creating, editing, and deleting any station.
- Implement admin functionality for viewing measurement data from all stations.
- Implement filtering of all measurement data by date and time.
- Implement deletion of measurement data from any station.
- Allow admins to create collections using measurement data from any station.
- Allow admins to rename and delete any collection.
- Ensure admins can only share collections they created themselves.

7. User Interface and UX Enhancements

- Implement a welcome page with a dashboard showing current measurement data.
- Create a user-friendly and visually appealing design.
- Implement responsive design for different screen sizes.
- Implement light/dark theme with persistent selection.
- Implement multilingual support with persistent language selection.
- Display measurement data using diagrams.
- Implement notifications (email or in-site).
- Implement a chat system between users.
- Implement station registration via QR code.

Login

Username

Password

Login

REPIFdb: repif

HomeStationsMeasurementsFriendsCollectionsSharedAccountLogout

Dashboard

Welcome, qwe qwe — role: User

My stations

2

Devices registered to your account

Open stations →

Measurements

5

Total measurements for your stations

Open measurements →

Collections

1

Collections created by you

Open collections →

Friends

1

People connected to your account

Open friends →

Shared with me

0

Collections your friends shared with you

Open shared collections →

REPIFdb: repif

HomeStationsMeasurementsFriendsCollectionsSharedAccountLogout

Measurements (My stations)

Station

(all my stations)

From (YYYY-MM-DD HH:MM:SS)

2026-01-01 00:00:00

To (YYYY-MM-DD HH:MM:SS)

2026-01-31 23:59:59

Filter

ID	Station	Timestamp	Temp	Hum	Press	Light	Gas
15	ST-1001	2026-01-19 20:57:35	22.80	42.90	1012.70	390.00	0.09
14	ST-1001	2026-01-19 20:47:35	22.40	43.50	1012.80	370.00	0.10
13	ST-1001	2026-01-19 20:37:35	22.10	44.00	1013.00	350.00	0.10
12	ST-1001	2026-01-19 20:27:35	21.70	44.90	1013.10	320.00	0.11
11	ST-1001	2026-01-19 20:17:35	21.40	45.20	1013.20	300.00	0.12

Limit 500 rows.

My Stations

Register a station (serial number)

Serial	Name	Description	Save
ST-1001	name	description	Save
ST-1002	name	description	Save

Friends

Add friend

Username	Name	Email	Remove
Userr	Firstt Lastt	Email@Email.Email	Remove

My Collections

Create collection from one station + time range

Name

Description

Station

(choose)▼

From (YYYY-MM-DD HH:MM:SS)

2026-01-01 00:00:00

To (YYYY-MM-DD HH:MM:SS)

2026-01-01 23:59:59

Create

ID	Name	Description	Actions	Share
				Shared with: Userr
				share to...▼
				Share
				unshare from...▼
				Unshare

1

test

View

test

Save

Delete

Collections shared with me

ID	Name	Description	Owner	View
1	test		qwe	Open

My account

Account settings

Username

qwee

Email

qwee@qwee.qwee

First name

qwee

Last name

qwee

New password (optional)

leave empty to keep old

Save changes

Admin - Users

Create new user

Username	Name	Email	Role	Delete
qwe	qwe qwe	qwe@qwe.qwe	User Save	Delete
qwee	qwee qwee	qwee@qwee.qwee	Admin Save	Delete
Userr	Firstt Lastt	Emaill@EmailL.Emaill	Admin Save	Delete

Admin - Stations

Create station

Serial	Name	Description	Owner (username or empty)	Save	Delete
ST-1001			qwe	<button>Save</button>	<button>Delete</button>
ST-1002			qwe	<button>Save</button>	<button>Delete</button>

Admin - Collections

Add measurement to a collection

collection id

measurement id

Add measurement ID

collection id

station...

▼

from 2026-01-01 00:00:00

to 2026-01-01 23:59:59

Add by range

ID	Name	Description	Creator	Rename		Delete
1	test		qwe	test	Save	Delete

Web Database Server Sprint

Maksym Bylym

Table of contents

1. Overview	3
2. Step 1: Ubuntu Server Preparation	3
2.1 System Update	3
2.2 Installing Docker Engine and Docker Compose	3
3. Step 2: Project Structure	3
4. Step 3: Docker Compose Configuration	4
5. Step 4: Webserver Dockerfile & Apache Configuration	5
5.1 Dockerfile	5
5.2 Apache Virtual Host Configuration	5
6. Step 5: File Permissions (Should)	6
7. Step 6: MySQL Security — mysql_secure_installation (Should)	6
7.1 Start the containers first	6
7.2 Run mysql_secure_installation inside the container	6
8. Step 7: Database and Test Users (Must)	7
8.1 Connect to MariaDB	7
8.2 Verify unix_socket Authentication (Must)	7
8.3 Verify Minimal Rights for pif_user (Should)	7
8.4 Create Test Tables and Users (Must)	7
9. Step 8: Test Website (Must)	7
10. Step 9: Launching and Verification	8
10.1 Build and Start Containers	8
10.2 Expected Output	8
10.3 Test Website Access (Must)	8
11. Step 10: phpMyAdmin (Should)	9
11.1 Accessing phpMyAdmin	9
11.2 Features Available	9
12. Step 11: HTTPS with Self-Signed Certificate (Could)	9
12.1 Generate Self-Signed Certificate	9
12.2 HTTP to HTTPS Redirect	9
12.3 Rebuild Containers with HTTPS	9

1. Overview

This document describes the installation and configuration of a web and database server as required by User Story L4 of the PIF (Portable Indoor Feedback) project. The solution uses two Docker containers running on Ubuntu Server: one for the Apache/PHP web server and one for the MariaDB database server. phpMyAdmin is also deployed for database management.

2. Step 1: Ubuntu Server Preparation

Before setting up the containers, the Ubuntu Server host must be updated and Docker must be installed.

2.1 System Update

The first command updates the package list, and the second upgrades all installed packages to their latest versions:

```
sudo apt update && sudo apt upgrade -y
```

2.2 Installing Docker Engine and Docker Compose

Docker is installed using the official Docker repository to ensure the latest version:

```
# Install prerequisites
sudo apt install -y ca-certificates curl gnupg lsb-release

# Add Docker GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Add Docker repository
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

# Allow current user to run Docker without sudo
sudo usermod -aG docker $USER && newgrp docker
```

3. Step 2: Project Structure

A dedicated directory is created to hold all configuration files and web content:

```
mkdir -p ~/pif-server/{webserver,database,phpmyadmin,mysql-data,www}
cd ~/pif-server
```


The resulting folder structure is:

```
~/pif-server/
├── docker-compose.yml      ← defines all containers
├── webserver/
│   ├── Dockerfile          ← builds the PHP/Apache image
│   └── apache-config/
│       └── 000-default.conf ← Apache virtual host config
├── www/                   ← website files (PHP, HTML, CSS)
├── mysql-data/            ← persistent MariaDB data
└── phpmyadmin/            ← optional phpMyAdmin config
```

4. Step 3: Docker Compose Configuration

Docker Compose defines all three containers in a single YAML file. The containers communicate with each other through a shared internal network called pif-network. The webserver and phpMyAdmin containers only start after the database is healthy.

```
# ~/pif-server/docker-compose.yml
version: '3.8'

services:
  # Container 1: MariaDB (Database Server)
  database:
    image: mariadb:11
    container_name: pif-database
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: pif_db
      MYSQL_USER: pif_user
      MYSQL_PASSWORD: pif_password
    volumes:
      - ./mysql-data:/var/lib/mysql
    networks:
      - pif-network
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Container 2: Apache + PHP (Web Server)
  webserver:
    build: ./webserver
    container_name: pif-webserver
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./www:/var/www/html
    depends_on:
      database:
        condition: service_healthy
    networks:
      - pif-network
    environment:
      DB_HOST: database
      DB_NAME: pif_db
```

```

    DB_USER: pif_user
    DB_PASSWORD: pif_password

# phpMyAdmin
phpmyadmin:
  image: phpmyadmin:latest
  container_name: pif-phpmyadmin
  restart: unless-stopped
  ports:
    - "8080:80"
  environment:
    PMA_HOST: database
    PMA_PORT: 3306
    MYSQL_ROOT_PASSWORD: rootpassword
  depends_on:
    database:
      condition: service_healthy
  networks:
    - pif-network

networks:
  pif-network:
    driver: bridge

```

5. Step 4: Webserver Dockerfile & Apache Configuration

5.1 Dockerfile

A custom Dockerfile builds the PHP 8.2 + Apache image with the required PHP extensions for database connectivity. PHP error display is enabled to assist with development.

```

# ~/pif-server/webserver/Dockerfile
FROM php:8.2-apache

# Install PHP extensions for database connectivity
RUN docker-php-ext-install mysqli pdo pdo_mysql

# Enable mod_rewrite for Apache
RUN a2enmod rewrite

# Copy Apache virtual host configuration
COPY apache-config/000-default.conf \
  /etc/apache2/sites-available/000-default.conf

# Set timezone
RUN echo "date.timezone = Europe/Luxembourg" >> \
  /usr/local/etc/php/php.ini

# Enable PHP error display (for development/testing)
RUN echo "display_errors = On" >> /usr/local/etc/php/php.ini
RUN echo "error_reporting = E_ALL" >> /usr/local/etc/php/php.ini

EXPOSE 80 443

```

5.2 Apache Virtual Host Configuration

The Apache virtual host configuration sets the document root and grants access to the web files directory. AllowOverride All enables .htaccess files for URL rewriting.

```
# ~/pif-server/webserver/apache-config/000-default.conf
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    <Directory /var/www/html>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

6. Step 5: File Permissions (Should)

To prevent unauthorised access to website files, correct ownership and permissions are set. The www-data user (Apache's user) is set as owner, and permissions are restricted so that only the owner can write to the files.

```
# Set ownership to the Apache web server user
sudo chown -R www-data:www-data ~/pif-server/www

# Directories: rwxr-xr-x (755), Files: rw-r--r-- (644)
sudo chmod -R 755 ~/pif-server/www

# Verify permissions
ls -la ~/pif-server/www/
```

7. Step 6: MySQL Security — mysql_secure_installation (Should)

After launching the containers, mysql_secure_installation is executed inside the MariaDB container to harden the database server. This removes insecure default settings.

7.1 Start the containers first

```
cd ~/pif-server
docker compose up -d
```

7.2 Run mysql_secure_installation inside the container

```
docker exec -it pif-database bash
mysql_secure_installation
```

8. Step 7: Database and Test Users (Must)

Connect to the MariaDB container and create the test database structure with test users:

8.1 Connect to MariaDB

```
docker exec -it pif-database mariadb -u root -p
# Enter: rootpassword
```

8.2 Verify unix_socket Authentication (Must)

```
SELECT User, Host, plugin FROM mysql.user WHERE User='root';
-- Should show plugin = 'unix_socket'
```

8.3 Verify Minimal Rights for pif_user (Should)

The dedicated database user for the website has only the minimum required privileges:

```
-- Restrict to only necessary operations
REVOKE ALL PRIVILEGES ON pif_db.* FROM 'pif_user'@'%';
GRANT SELECT, INSERT, UPDATE, DELETE ON pif_db.* TO 'pif_user'@'%';
FLUSH PRIVILEGES;

-- Verify the result
SHOW GRANTS FOR 'pif_user'@'%';
```

8.4 Create Test Tables and Users (Must)

```
USE pif_db;

-- Users table
CREATE TABLE users (
    username VARCHAR(50) PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(200) NOT NULL,
    is_admin TINYINT(1) DEFAULT 0
);

-- Insert test users
INSERT INTO users VALUES
    ('admin', 'Admin User', 'admin@pif.lu', SHA2('admin123', 256), 1),
    ('user1', 'Test User One', 'user1@pif.lu', SHA2('user123', 256), 0),
    ('user2', 'Test User Two', 'user2@pif.lu', SHA2('user123', 256), 0);

SELECT username, full_name, is_admin FROM users;
EXIT;
```

9. Step 8: Test Website (Must)

A PHP test page is created in the web directory to verify that the web server is running and that it can successfully connect to the database.

```
# ~/pif-server/www/index.php (key excerpt)
<?php
$host = getenv('DB_HOST') ?: 'database';
$db    = getenv('DB_NAME')  ?: 'pif_db';
$user  = getenv('DB_USER')  ?: 'pif_user';
$pass  = getenv('DB_PASSWORD') ?: 'pif_password';

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db;charset=utf8", $user, $pass);
    $status = "Database connection successful!";
} catch (PDOException $e) {
    $status = "Error: " . $e->getMessage();
}
?>
<!-- HTML renders database connection status and test user table -->
```

10. Step 9: Launching and Verification

After all configuration files are in place, the containers are built and started:

10.1 Build and Start Containers

```
cd ~/pif-server
docker compose up -d --build

# Check container status
docker compose ps
```

10.2 Expected Output

NAME	STATUS	PORTS
pif-database	running	3306/tcp
pif-webserver	running	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp
pif-phpmyadmin	running	0.0.0.0:8080->80/tcp

10.3 Test Website Access (Must)

```
# Retrieve server IP address
hostname -I

# Test from the command line
curl http://localhost
curl http://localhost/test.php

# Open in browser:
# http://<server-ip>          -> Main page with DB connection status
# http://<server-ip>/test.php -> PHP information page
# http://<server-ip>:8080     -> phpMyAdmin
```

11. Step 10: phpMyAdmin (Should)

phpMyAdmin is deployed as a Docker container using the official phpMyAdmin image. It connects to the MariaDB container through the internal pif-network.

11.1 Accessing phpMyAdmin

- Open browser and navigate to: `http://<server-ip>:8080`
- Login with root credentials: username root, password rootpassword
- Alternatively, login as the website user: username pif_user, password pif_password

11.2 Features Available

- Browse and manage the pif_db database
- View, insert, edit and delete records in the users table
- Run arbitrary SQL queries
- Import and export database dumps

12. Step 11: HTTPS with Self-Signed Certificate (Could)

A self-signed SSL certificate is generated and the Apache virtual host is configured for HTTPS. HTTP traffic is automatically redirected to HTTPS.

12.1 Generate Self-Signed Certificate

```
mkdir -p ~/pif-server/webserver/ssl

openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout ~/pif-server/webserver/ssl/server.key \
  -out ~/pif-server/webserver/ssl/server.crt \
  -subj "/C=LU/ST=Luxembourg/L=Luxembourg/O=PIF/CN=pif-server"
```

12.2 HTTP to HTTPS Redirect

The HTTP virtual host is updated to redirect all traffic to HTTPS:

```
<VirtualHost *:80>
  RewriteEngine On
  RewriteCond %{HTTPS} off
  RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
</VirtualHost>
```

12.3 Rebuild Containers with HTTPS

```
docker compose down
docker compose up -d --build
```

L5 — Data Transfer to Calendar

Maksym Bylym

Contents

1. Overview.....	3
2. System Architecture.....	3
3. Step 1: Creating the Measurements Table	3
4. Step 2: Creating the PHP Data Transfer Script.....	4
5. Step 3: Setting Permissions and Rebuilding the Container	12
6. Problem 1: HTTP to HTTPS Redirect.....	12
7. Step 4: Testing the Script	13
8. Step 5: Manual Entry Form.....	13
9. Problem 2: Station Sending to Wrong IP	14
10. Problem 3: Field Name Mismatch	15
11. Final Verification — Live Data Transfer	15
12. Requirements Checklist.....	16
13. Website Deployment and Integration	17
13.1 Copying the Website Files	18
13.2 Database Configuration	18
13.3 Updating config.php for Docker.....	18
13.4 Registering the Station.....	18
13.5 Updating the Station Configuration	18
13.6 Problem: Timestamp Format Mismatch	19
13.7 Live Data on the Website	19
13.8 Database Verification	21

1. Overview

The goal was to create a PHP script that receives sensor measurement data from the station Raspberry Pi via HTTP POST and stores it in the MariaDB database running on the Ubuntu Server from the previous sprint (L4).

The solution builds directly on the infrastructure set up in L4: the same Docker containers (pif-webserver, pif-database), the same pif_db database, and the same project directory ~/pif-server/www/ are used. The PHP script also includes a manual entry form for testing purposes.

2. System Architecture

The system consists of two separate machines connected on the same local network:

- Station Raspberry Pi (hostname: bylma731) — runs measure.py which reads sensor values and sends them via HTTP POST to the server.
- Ubuntu Server (hostname: REPIF) — runs the Docker containers from L4. The new data_transfer.php script is placed in ~/pif-server/www/ and is served by the pif-webserver Apache container on port 80.

The station reads the destination IP and path from a configuration file located at /opt/station/config.toml. After updating this file to point to the Ubuntu Server, the station sends measurements automatically every 2 seconds.

3. Step 1: Creating the Measurements Table

The pif_db database already existed from the L4 sprint, but it only contained a users table. I needed to create a new measurements table to store the sensor data sent by the stations.

I connected to the MariaDB container using the following command on the Ubuntu Server:

```
docker exec -it pif-database mariadb -u root -p
```

After entering the root password (rootpassword), I created the table with the following SQL:

```

USE pif_db;

CREATE TABLE measurements (
    id INT AUTO_INCREMENT PRIMARY
KEY,    serial_number VARCHAR(50) NOT NULL,
timestamp DATETIME NOT NULL,
temperature FLOAT,    humidity FLOAT,
pressure FLOAT,    light FLOAT,
    gas FLOAT,    created_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
);

GRANT SELECT, INSERT, UPDATE, DELETE ON pif_db.measurements TO
'pif_user'@'%';

FLUSH PRIVILEGES;

```

The table stores the station serial number, timestamp, and all five sensor values: temperature (°C), humidity (%), pressure (hPa), light (lux), and gas/CO2 (ppm). The created_at field is filled automatically by the database.

```

MariaDB [pif_db]> SHOW TABLES;
+-----+
| Tables_in_pif_db |
+-----+
| measurements      |
| users              |
+-----+
2 rows in set (0.000 sec)

MariaDB [pif_db]> DESCRIBE measurements;
+-----+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default      | Extra      |
+-----+-----+-----+-----+-----+-----+-----+
| id         | int(11)   | NO   | PRI | NULL         | auto_increment |
| serial_number | varchar(50) | NO   |     | NULL         |
| timestamp  | datetime  | NO   |     | NULL         |
| temperature | float     | YES  |     | NULL         |
| humidity    | float     | YES  |     | NULL         |
| pressure    | float     | YES  |     | NULL         |
| light       | float     | YES  |     | NULL         |
| gas         | float     | YES  |     | NULL         |
| created_at | timestamp | YES  |     | current_timestamp() |
+-----+-----+-----+-----+-----+-----+-----+
9 rows in set (0.002 sec)

```

4. Step 2: Creating the PHP Data Transfer Script

I created the PHP script at ~/pif-server/www/data_transfer.php on the Ubuntu Server. The script serves two purposes:

- Receives sensor data via HTTP POST from the measurement stations and saves it to the database.
- Provides a manual entry form with validation for testing purposes.

The script uses environment variables for database credentials (DB_HOST, DB_NAME, DB_USER, DB_PASSWORD), which are already defined in the dockercompose.yml from L4. This means no hardcoded passwords are needed.

The script detects whether a request comes from a browser or from a station by checking the HTTP Accept header. Browser requests receive an HTML form response, while station requests receive a JSON response.

Input validation checks that the serial number and timestamp are present and that the timestamp format is valid. Numeric sensor values are validated as floats, and null is stored if a value is missing or invalid.

```
<?php
```

```
/**
```

```
 * data_transfer.php
```

```
 * Portable Indoor Feedback — L5 Data Transfer Script
```

```
 *
```

```
 * Accepts sensor data via HTTP POST from measurement stations * and stores it in  
   the pif_db MariaDB database.
```

```
 * Also provides a manual entry form for testing.
```

```
 *
```

```
 * Database credentials are read from environment variables
```

```
 * set in docker-compose.yml (DB_HOST, DB_NAME, DB_USER, DB_PASSWORD).
```

```
 */
```

```
// --- Database configuration (uses Docker environment variables) ---
```

```
$db_host = getenv('DB_HOST') ?: 'database';
```

```
$db_name = getenv('DB_NAME') ?: 'pif_db';
```

```
$db_user = getenv('DB_USER') ?: 'pif_user';
```

```
$db_pass = getenv('DB_PASSWORD') ?: 'pif_password';
```

```
/**
```

```
 * Connect to the MariaDB database using PDO.
```

```
 * Returns a PDO instance or sends a JSON error response and exits.
```

```
 */
```

```
function getDB($host, $name, $user, $pass) {
```

```
try {
```

```
    $pdo = new PDO("mysql:host=$host;dbname=$name;charset=utf8", $user,  
$pass);
```

```
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```
return $pdo;
```

```

    } catch (PDOException $e) {
http_response_code(500);
        die(json_encode(['status' => 'error', 'message' => 'DB connection failed: ' .
        $e->getMessage()]));
    }
}

/**
 * Validate and sanitize a numeric (float) input.
 * Returns the float value, or null if invalid/missing.
 */
function validateFloat($value) {
    if ($value === null || $value === "") return null;
    $v = filter_var($value, FILTER_VALIDATE_FLOAT);
    return $v !== false ? $v : null;
}

// --- Handle HTTP POST request ---
$message = "";
$message_type = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // Read and sanitize inputs
    $serial    = htmlspecialchars(trim($_POST['station_serial'] ?? ""));
    $raw_ts    = trim($_POST['timestamp'] ?? "");
    $timestamp = htmlspecialchars($raw_ts);
    $temp      = validateFloat($_POST['temperature'] ?? null);
    $humidity  = validateFloat($_POST['humidity'] ?? null);
    $co2       = validateFloat($_POST['gas'] ?? null);    // station sends as 'gas'
    $pressure  = validateFloat($_POST['pressure'] ?? null);
    $light     = validateFloat($_POST['light'] ?? null);

    // --- Validation ---
    $errors = [];

```

```

    if (empty($serial)) $errors[] = 'Serial number is required.';
    if (empty($timestamp)) $errors[] = 'Timestamp is required.';
    if (!empty($timestamp) && !strtotime($timestamp))
        $errors[] = 'Timestamp format invalid. Use: YYYY-MM-DD HH:MM:SS';

    if (empty($errors)) {
        $pdo = getDB($db_host, $db_name, $db_user, $db_pass);

        // Insert measurement record into database
        $stmt = $pdo->prepare("
            INSERT INTO measurements
                (serial_number, timestamp, temperature, humidity, pressure, light, gas)
VALUES
                (:serial, :timestamp, :temperature, :humidity, :pressure, :light, :gas)
        ");
        $stmt->execute([
            ':serial' => $serial,
            ':timestamp' => $timestamp,
            ':temperature' => $temp,
            ':humidity' => $humidity,
            ':pressure' => $pressure,
            ':light' => $light,
            ':gas' => $co2,
        ]);

        // If request comes from a station (not a browser), return JSON response
        $accept = $_SERVER['HTTP_ACCEPT'] ?? '';
        if (strpos($accept, 'text/html') === false) {
            header('Content-Type: application/json');
            echo json_encode(['status' => 'success', 'message' => 'Measurement saved.',
            'id' => $pdo->lastInsertId()]);
            exit;
        }

        $message = 'Data saved successfully!';
    }

```

```

$message_type = 'success';

} else {
    // If station request failed validation, return JSON error
    $accept = $_SERVER['HTTP_ACCEPT'] ?? '';    if
    (strpos($accept, 'text/html') === false) {
        http_response_code(400);

        header('Content-Type: application/json');
        echo json_encode(['status' => 'error', 'errors' => $errors]);
    exit;
    }
    $message = implode('<br>', $errors);
    $message_type = 'error';
}
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>PIF – Data Transfer</title>
    <style>
        * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
        font-family: 'Segoe UI', Arial, sans-serif;
        background: #f0f2f5;        display: flex;
        justify-content: center;        padding: 40px
        16px;
    }
    .container {
        background: white;
        border-radius: 10px;

```

```

        box-shadow: 0 4px 16px rgba(0,0,0,0.1);
padding: 32px;          width: 100%;
max-width: 560px;
    }
    h1 { font-size: 1.4rem; color: #1a1a2e; margin-bottom: 4px; }
    h2 { font-size: 1rem; color: #555; font-weight: normal; margin-bottom: 24px; }
label {

    display: block;
margin-top: 16px;      font-
size: 0.85rem;        font-
weight: 600;          color:
#333;                 margin-bottom:
4px;
    }
    input[type=text], input[type=datetime-local] {
width: 100%;          padding: 10px 12px;
border: 1px solid #ddd;      border-radius:
6px;      font-size: 0.95rem;
transition: border-color 0.2s;
    }
    input:focus { outline: none; border-color: #0077cc; }
.hint { font-size: 0.78rem; color: #888; margin-top: 3px; }
button {      margin-top: 24px;      width: 100%;
padding: 12px;      background: #0077cc;      color:
white;      border: none;      border-radius: 6px;
font-size: 1rem;      cursor: pointer;      transition:
background 0.2s;
    }
    button:hover { background: #005fa3; }
.alert {      padding:
12px 16px;      border-
radius: 6px;      margin-
bottom: 20px;      font-
size: 0.9rem;

```

```

    }
    .success { background: #e6f4ea; color: #2d6a4f; border: 1px solid #b7dfbf; }
    .error { background: #fdecea; color: #a61c00; border: 1px solid #f5c6c0; }
    .grid { display: grid; grid-template-columns: 1fr 1fr; gap: 0 16px; }
hr { border: none; border-top: 1px solid #eee; margin: 24px 0 20px; }
</style>
</head>
<body>
<div class="container">
    <h1> Portable Indoor Feedback</h1>
    <h2>Manual Data Entry — L5 Data Transfer</h2>

    <?php if ($message): ?>
        <div class="alert <?= $message_type ?>"><?= $message ?></div>
    <?php endif; ?>

    <form method="POST" action="data_transfer.php">

        <label for="serial_number">Serial Number <span
style="color:red">*</span></label>
        <input type="text" id="serial_number" name="serial_number"
required placeholder="e.g. STATION01"
        value="<?= htmlspecialchars($_POST['serial_number'] ?? '') ?>">

        <label for="timestamp">Timestamp <span style="color:red">*</span></label>
        <input type="datetime-local" id="timestamp" name="timestamp"
required
        value="<?= htmlspecialchars(isset($_POST['timestamp']) ? str_replace(' ',
'T', $_POST['timestamp']) : '') ?>">
        <p class="hint">Date and time of measurement</p>

    <hr>

    <div class="grid">
        <div>
            <label for="temperature">Temperature</label>

```



```
<input type="text" id="temperature" name="temperature"
placeholder="e.g. 20.00"
value="<?= htmlspecialchars($_POST['temperature'] ?? ") ?>">
<p class="hint">°C</p>
</div>
<div>
<label for="humidity">Humidity</label>
<input type="text" id="humidity" name="humidity"
placeholder="e.g. 50.00"
value="<?= htmlspecialchars($_POST['humidity'] ?? ") ?>">
<p class="hint">%</p>
</div>
<div>
<label for="pressure">Pressure</label>
<input type="text" id="pressure" name="pressure"
placeholder="e.g. 1013.25"
value="<?= htmlspecialchars($_POST['pressure'] ?? ") ?>">
<p class="hint">hPa</p>
</div>
<div>
<label for="light">Light</label>
<input type="text" id="light" name="light"
placeholder="e.g. 350.00"
value="<?= htmlspecialchars($_POST['light'] ?? ") ?>">
<p class="hint">lux</p>
</div>
</div>

<label for="gas">Gas (CO2/Air Quality)</label>
<input type="text" id="gas" name="gas"
placeholder="e.g. 500.00"
value="<?= htmlspecialchars($_POST['gas'] ?? ") ?>">
<p class="hint">ppm</p>

<button type="submit">Submit Measurement</button>
```

```
</form>
</div>
</body>
</html>
```

5. Step 3: Setting Permissions and Rebuilding the Container

After creating the script, I set the correct file permissions so the Apache web server could read it:

```
sudo chown -R www-data:www-data ~/pif-server/www
sudo chmod -R 755 ~/pif-server/www
```

Since the `www/` directory is mounted as a Docker volume, the file is immediately available inside the container without rebuilding. However, I had a problem with the Apache configuration at this point.

6. Problem 1: HTTP to HTTPS Redirect

When I tested the script using `curl`, I received a 301 Moved Permanently redirect instead of a success response:

```
d | curl -X POST http://localhost/data_transfer.php -
    "serial_number=STATION01" ... The response was:
    <h1>Moved Permanently</h1>
    <p>The document has moved <a
    href="https://localhost/data_transfer.php">here</a>.</p>
```

I tried using HTTPS with the `-k` flag to ignore the self-signed certificate warning, but got another error:

```
curl: (35) OpenSSL SSL_connect: SSL_ERROR_SYSCALL in connection to
localhost:443
```

To diagnose the issue, I checked the Apache configuration inside the running container:

```
docker exec pif-webserver cat /etc/apache2/sites-
enabled/000default.conf
```

This revealed that the container still had the old HTTP-to-HTTPS redirect rule from L4 baked into its image, even though I had already removed it from the config file on disk. The container had never been rebuilt after the change.

The fix was to make sure the config file on disk had no redirect, then rebuild the container:

```
cd ~/pif-server docker
compose down docker compose
up -d --build
```

```

bylma731@REPIF:~/pif-server$ cd ~/pif-server
bylma731@REPIF:~/pif-server$ docker compose down
WARN[0000] /home/bylma731/pif-server/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 4/4
  Container pif-phpmyadmin      Removed
  Container pif-webserver       Removed
  Container pif-database        Removed
  Network pif-server_pif-network Removed
bylma731@REPIF:~/pif-server$ docker compose up -d --build

```

ng

7. Step 4: Testing the Script

After rebuilding the container, I tested the script by sending a simulated POST request using curl:

```

curl -X POST http://localhost/data_transfer.php \
  -d "serial_number=STATION01" \
  -d "timestamp=2026-03-11 10:30:00" \
  -d "temperature=20.0" \
  -d "humidity=50.0" \
  -d "pressure=1013.0" \
  -d "light=350.0" \
  -d "gas=500.0"

```

The response was:

```

{"status":"success","message":"Measurement saved.", "id":"1"}

```

I then verified the data was stored correctly in the database:

```

docker exec -it pif-database mariadb -u root -p
USE pif_db;
SELECT * FROM measurements;

```

8. Step 5: Manual Entry Form

The PHP script also includes a manual entry form accessible via a browser. The form allows manual entry of all measurement fields and validates the input before saving. Required fields (serial number and timestamp) are marked with a red asterisk. Sensor value fields accept decimal numbers and display the unit (°C, %, hPa, lux, ppm) as hints.

The form can be accessed at:

```

http://<server-ip>/data_transfer.php

```

The screenshot shows a web browser window with the address bar displaying "Not secure 192.168.4.16/data_transfer.php". The browser's address bar also shows "Summarize" and "A" icons. The browser's tabs bar shows "Навчання", "відео і музика", "інструменти", "стажування", and "Other favorites". The main content area displays a form titled "Portable Indoor Feedback" with the subtitle "Manual Data Entry — L5 Data Transfer". The form contains the following fields:

- Serial Number ***: A text input field with the placeholder "e.g. STATION01".
- Timestamp ***: A date and time input field with the placeholder "mm/dd/yyyy --:-- --" and a calendar icon.
- Temperature**: A text input field with the placeholder "e.g. 20.00" and a unit label "°C" below it.
- Humidity**: A text input field with the placeholder "e.g. 50.00" and a unit label "%" below it.
- Pressure**: A text input field with the placeholder "e.g. 1013.25" and a unit label "hPa" below it.
- Light**: A text input field with the placeholder "e.g. 350.00" and a unit label "lux" below it.
- Gas (CO₂/Air Quality)**: A text input field with the placeholder "e.g. 500.00" and a unit label "ppm" below it.

At the bottom of the form is a large blue button labeled "Submit Measurement".

9. Problem 2: Station Sending to Wrong IP

After the server was working, I needed to connect the station Raspberry Pi to send its real sensor data to the server. I first confirmed the station could reach the server over the network:

```
ping 192.168.4.69
```

The ping was successful with no packet loss. I then checked that `measure.py` was running on the station:

```
pgrep -a measure.py
```

It was running but with the `--offline` flag, meaning it was not sending data to any server. I needed to update the destination in the configuration file and restart the script without that flag.

There were multiple `config.toml` files on the station. I found all of them with:

```
find / -name 'config.toml' 2>/dev/null
```

The files found were:

- `/opt/station/config.toml` — the correct one read by `measure.py`
- `/opt/station/www/config.toml` — a copy used by the local dashboard
- `/var/station/config.toml` — another copy
- `/home/bylma731/Downloads/ressourcen/Ressourcen/Firmware und Scripts/config.toml` — original download I edited the correct file:

```
sudo nano /opt/station/config.toml
```

And

updated the destination section:

```
[destination] ip =  
"192.168.4.69" path =  
"/data_transfer.php"
```

10. Problem 3: Field Name Mismatch

After updating the config and restarting `measure.py`, the station was connecting to the server but no new rows appeared in the database. I checked the Apache logs on the server:

```
docker logs pif-webserver -f
```

POST requests were arriving from the station IP, but the data was not being saved. I investigated how `measure.py` sends its data by reading the source code:

```
grep -A 10 'destination\|http\|post\|send\|request'  
/opt/station/measure.py
```

This revealed that `measure.py` sends the station identifier as `station_serial`, not `serial_number` as I had used in the PHP script. The field name mismatch meant the serial number was always empty, causing validation to fail silently.

I fixed this by updating the PHP script on the server to read the correct field name:

```
// Before (wrong):  
$serial = htmlspecialchars(trim($_POST['serial_number'] ?? ''));  
  
// After (correct):  
$serial = htmlspecialchars(trim($_POST['station_serial'] ??  
''));
```

Since `www/` is a mounted volume, the change took effect

immediately without rebuilding the container.

11. Final Verification — Live Data Transfer

After fixing the field name, I restarted `measure.py` on the station without the `--offline` flag:

```
python3 /opt/station/measure.py > /tmp/station/result.csv & |
```

then monitored the database on the server in real time:

```
watch -n 3 'docker exec pif-database mariadb -u root -prootpassword  
pif_db -e "SELECT * FROM measurements ORDER BY created_at DESC LIMIT 5;"'
```

New rows with real sensor values started appearing every 2 seconds, confirming that the full data transfer pipeline was working correctly from the physical sensors through the station Raspberry Pi to the MariaDB database on the Ubuntu Server.

12. Requirements Checklist

Must

- PHP script for data transfer has been created (data_transfer.php).
- Serial number, timestamp, and all measured values (temperature, humidity, pressure, light, gas) are transferred correctly and stored in the database.
- The data transfer script is accessible on the installed web and database server.
- Annotated source code — all functions and sections in the PHP script are commented.
- User documentation — this document explains how the script works.

Should

- The PHP script contains a form for manual entry of measurement data.
- The form has an appealing design and is user-friendly.
- Entered data is validated (required fields, float validation, timestamp format check).

```
Every 3.0s: docker exec pif-database mariadb -u root -prootpassword pif_db -e "SELECT * FROM measurements ORDER BY created_at DESC LIMIT 5;" REPIF: Thu Mar 19 15:17:59 2026
```

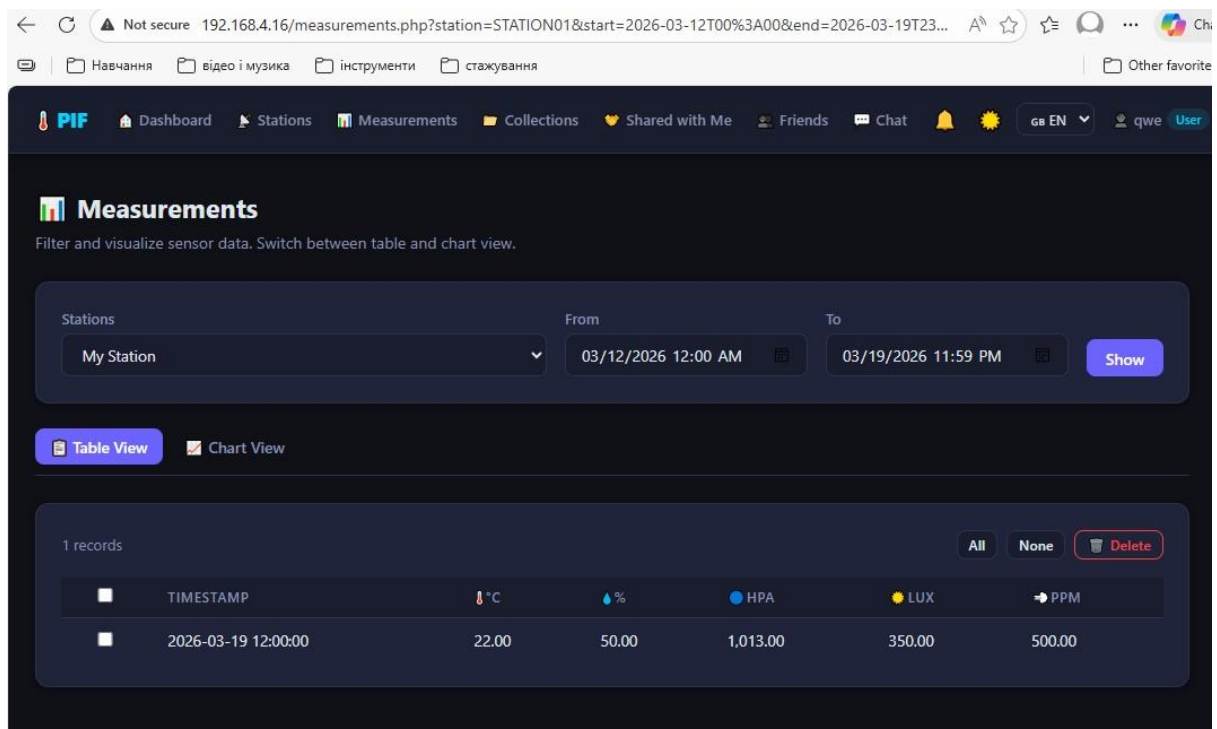
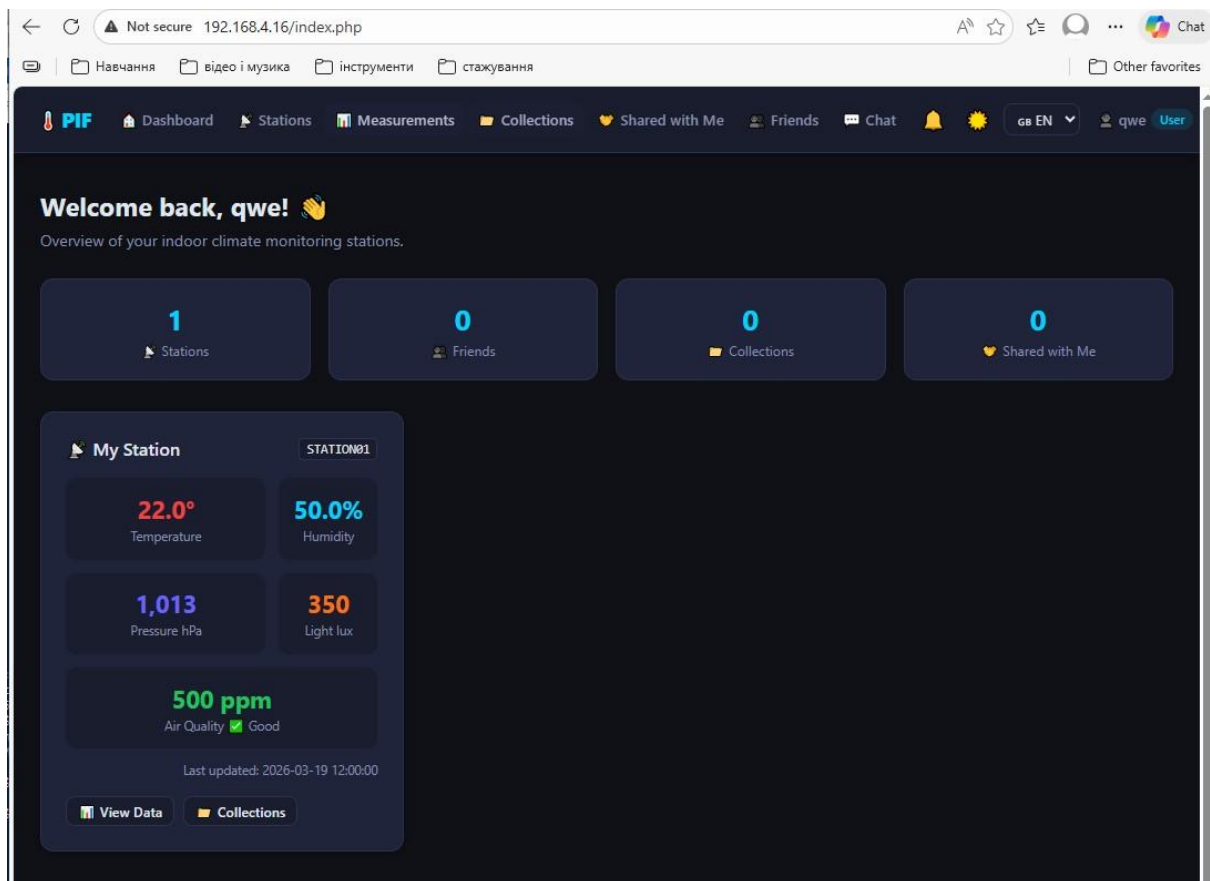
id	serial_number	timestamp	temperature	humidity	pressure	light	gas	created_at
2710	STATION01	2026-03-19 15:17:58	25.3999	0	993.631	9.16667	1958	2026-03-19 14:17:58
2709	STATION01	2026-03-19 15:17:55	25.3977	0	993.624	9.16667	1958	2026-03-19 14:17:55
2708	STATION01	2026-03-19 15:17:53	25.3951	0	993.626	9.16667	1947	2026-03-19 14:17:53
2707	STATION01	2026-03-19 15:17:51	25.3973	0	993.617	9.16667	1960	2026-03-19 14:17:51
2706	STATION01	2026-03-19 15:17:49	25.3938	0	993.641	9.16667	1959	2026-03-19 14:17:49

```
Every 3.0s: docker exec pif-database mariadb -u root -prootpassword pif_db -e "SELECT * FROM measurements ORDER BY created_at DESC LIMIT 5;" REPIF: Thu Mar 19 15:18:12 2026
```

id	serial_number	timestamp	temperature	humidity	pressure	light	gas	created_at
2716	STATION01	2026-03-19 15:18:10	25.4031	0	993.623	9.16667	1983	2026-03-19 14:18:10
2715	STATION01	2026-03-19 15:18:08	25.4069	0	993.609	9.16667	1977	2026-03-19 14:18:08
2714	STATION01	2026-03-19 15:18:06	25.4053	0	993.62	9.16667	1971	2026-03-19 14:18:06
2713	STATION01	2026-03-19 15:18:04	25.4056	0	993.615	9.16667	1950	2026-03-19 14:18:04
2712	STATION01	2026-03-19 15:18:02	25.4063	0	993.626	9.16667	1958	2026-03-19 14:18:02

```
Every 3.0s: docker exec pif-database mariadb -u root -prootpassword pif_db -e "SELECT * FROM measurements ORDER BY created_at DESC LIMIT 5;" REPIF: Thu Mar 19 15:18:18 2026
```

id	serial_number	timestamp	temperature	humidity	pressure	light	gas	created_at
2719	STATION01	2026-03-19 15:18:16	25.4047	0	993.603	9.16667	1997	2026-03-19 14:18:16
2718	STATION01	2026-03-19 15:18:14	25.4028	0	993.617	9.16667	1996	2026-03-19 14:18:14
2717	STATION01	2026-03-19 15:18:12	25.4043	0	993.634	9.16667	1990	2026-03-19 14:18:12
2716	STATION01	2026-03-19 15:18:10	25.4031	0	993.623	9.16667	1983	2026-03-19 14:18:10
2715	STATION01	2026-03-19 15:18:08	25.4069	0	993.609	9.16667	1977	2026-03-19 14:18:08



13. Website Deployment and Integration

After the data transfer pipeline was working with `data_transfer.php`, the next step was to deploy the full PIF website (developed in User Story L3) onto the same web and database server. This allows

users to log in, view their stations, browse measurement data, and use all other website features directly from the server.

13.1 Copying the Website Files

The website files were transferred from the development machine to the Ubuntu Server using scp. All PHP files were copied into the existing ~/pif-server/www/ directory, which is mounted as a Docker volume into the Apache container.

```
scp pif_website.zip bylma731@192.168.4.16:~/ cd
~ && unzip pif_website.zip
sudo cp ~/pif_website/*.php ~/pif-server/www/ sudo
cp ~/pif_website/*.sql ~/pif-server/www/
```

After copying, file permissions were set correctly for the Apache web server user:

```
sudo chown -R www-data:www-data ~/pif-server/www
sudo chmod -R 755 ~/pif-server/www
```

Since the www/ directory is a mounted Docker volume, the new files were immediately available inside the container without rebuilding.

13.2 Database Configuration

The website uses a separate database called portableindoorfeedback with a more complete schema than the simple measurements table from data_transfer.php. This database includes tables for users, stations, measurements, collections, friendships, and more. The database schema was imported into the MariaDB container:

```
docker exec -i pif-database mariadb -u root -prootpassword <
~/pif_website/portableindoorfeedback.sql docker exec -i pif-
database mariadb -u root -prootpassword <
~/pif_website/schema_extensions.sql
```

The pif_user database user was granted access to the new database:

```
docker exec pif-database mariadb -u root -prootpassword -e "GRANT SELECT, INSERT,
UPDATE, DELETE ON portableindoorfeedback.* TO 'pif_user'@'%'; FLUSH PRIVILEGES;"
```

13.3 Updating config.php for Docker

The website's config.php originally used localhost and root credentials for local development. This had to be updated to use the Docker container environment variables:

```
define('DB_HOST', 'database'); // Docker container name
define('DB_NAME', 'portableindoorfeedback'); define('DB_USER',
'pif_user'); define('DB_PASS', 'pif_password');
define('DB_CHARSET', 'utf8mb4');
```

Inside Docker, the database host is "database" (the container name defined in docker-compose.yml), not "localhost".

13.4 Registering the Station

The website's receive_data.php script checks that the station exists in the database before accepting measurements. The station had to be registered in the portableindoorfeedback database and assigned to a user account:

```
docker exec pif-database mariadb -u root -prootpassword portableindoorfeedback -e
"INSERT INTO station (pk_serialNumber, name) VALUES ('STATION01', 'My Station');"
docker exec pif-database mariadb -u root -prootpassword portableindoorfeedback -e
"UPDATE station SET fk_user_owns='qwe' WHERE pk_serialNumber='STATION01';"
```

13.5 Updating the Station Configuration

The station Raspberry Pi's config.toml was updated to point to the website's receive_data.php script instead of data_transfer.php. The server IP was also updated to the current address:


```
[destination] ip =  
"192.168.4.16" path =  
"/receive_data.php"
```

13.6 Problem: Timestamp Format Mismatch

After updating the configuration and restarting measure.py, the station was sending POST requests to the server but receiving HTTP 400 errors. Checking the Apache logs showed:

```
192.168.4.94 - - [19/Mar/2026:14:55:51 +0000] "POST /receive_data.php HTTP/1.1" 400 223
```

The problem was that measure.py sends timestamps with microseconds (e.g., 2026-03-19 15:56:17.301718), but the website's receive_data.php only accepted the format Y-m-d H:i:s (without microseconds). The DateTime validation was failing and returning a 400 error.

The fix was to add the microsecond format as the first option in the timestamp parsing: //

Before (line 28):

```
$dt = DateTime::createFromFormat('Y-m-d H:i:s',$timestamp) ?:  
DateTime::createFromFormat('Y-m-d\TH:i:s',$timestamp);
```

// After:

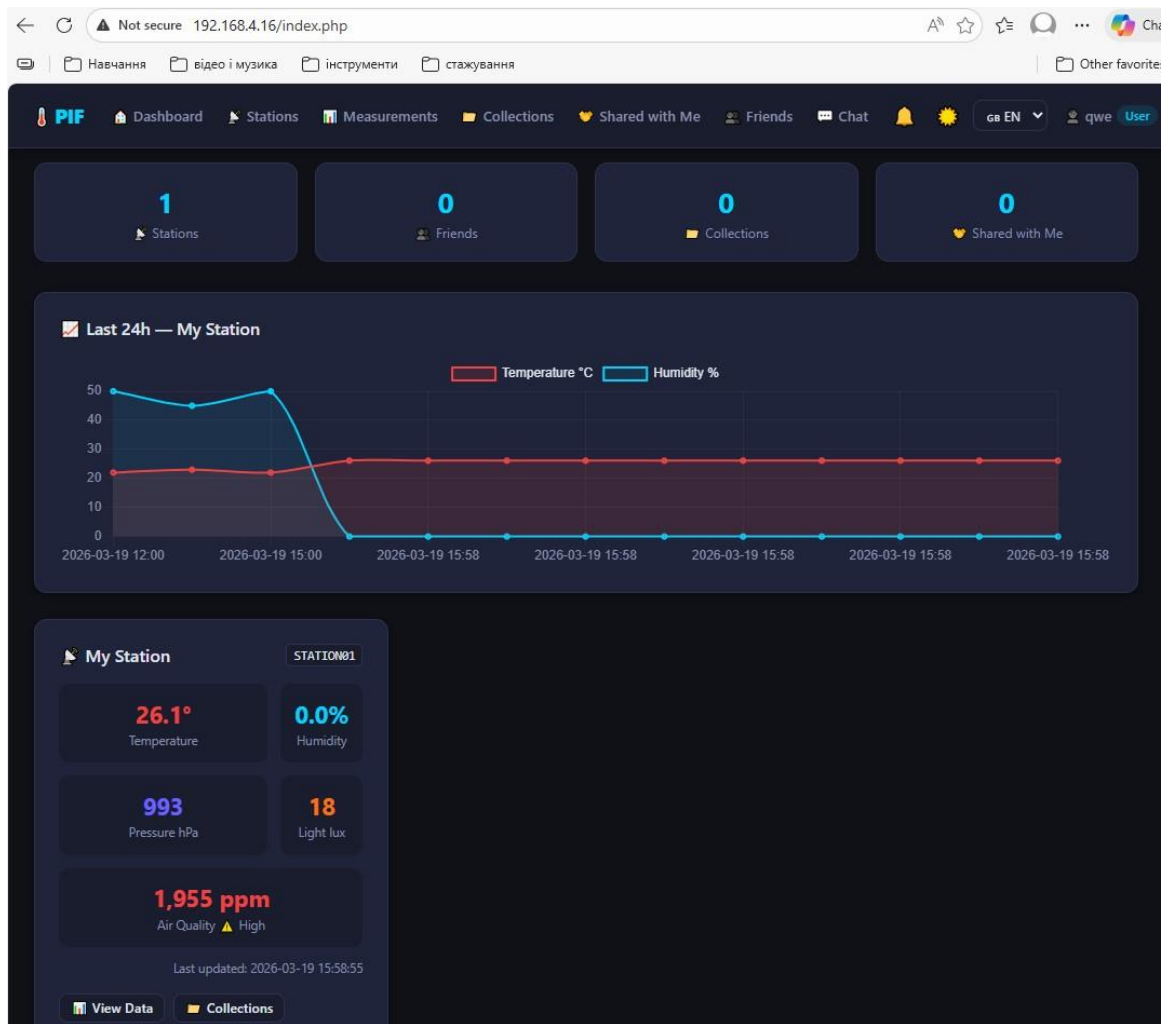
```
$dt = DateTime::createFromFormat('Y-m-d H:i:s.u',$timestamp) ?:  
DateTime::createFromFormat('Y-m-d H:i:s',$timestamp) ?:  
DateTime::createFromFormat('Y-m-d\TH:i:s',$timestamp);
```

After this fix, the Apache logs immediately showed HTTP 200 responses and data started flowing into the portableindoorfeedback database:

```
192.168.4.94 - - [19/Mar/2026:14:58:37 +0000] "POST /receive_data.php HTTP/1.1" 200 247
```

13.7 Live Data on the Website

With the data pipeline fully working, the website dashboard shows real-time sensor data from the station. The dashboard displays the latest temperature, humidity, pressure, light, and gas/CO2 values, along with a 24-hour chart of temperature and humidity trends.



The Measurements page allows users to select a station and a date range to view historical measurement data in a table. Users can switch between a table view and a chart view.

← ↻ 🔒 Not secure 192.168.4.16/measurements.php?station=STATION01&start=2026-03-01T00%3A00&end=2026-03-19T23... ⚙️ ☆ ⌵ ⌵ ⌵ Cha

Навчання відео і музика інструменти стажування Other favorites

PIF Dashboard Stations Measurements Collections Shared with Me Friends Chat 📢 🌞 ga EN 👤 qwe User

Measurements

Filter and visualize sensor data. Switch between table and chart view.

Stations From To

My Station 03/01/2026 12:00 AM 03/19/2026 11:59 PM Show

Table View Chart View

21 records All None Delete

	TIMESTAMP	°C	%	HPA	LUX	PPM
🗕	2026-03-19 15:59:11	26.09	0.00	993.05	17.50	1,964.00
🗕	2026-03-19 15:59:09	26.08	0.00	993.04	17.50	1,962.00
🗕	2026-03-19 15:59:07	26.09	0.00	993.05	17.50	1,960.00
🗕	2026-03-19 15:59:05	26.09	0.00	993.06	17.50	1,960.00
🗕	2026-03-19 15:59:03	26.09	0.00	993.05	17.50	1,959.00
🗕	2026-03-19 15:59:01	26.10	0.00	993.05	17.50	1,957.00
🗕	2026-03-19 15:58:59	26.10	0.00	993.06	17.50	1,957.00
🗕	2026-03-19 15:58:57	26.10	0.00	993.06	17.50	1,955.00
🗕	2026-03-19 15:58:55	26.10	0.00	993.06	17.50	1,955.00
🗕	2026-03-19 15:58:53	26.10	0.00	993.06	16.67	1,953.00

13.8 Database Verification

To verify the data pipeline, I monitored the database on the server in real time using the watch command. New rows with real sensor values appeared every 2 seconds in the portableindoorfeedback measurement table:

```
Every 3.0s: docker exec pif-database mariadb -u root -prootpassword pif_db -e "SELECT * FROM measurements ORDER BY created_at DESC LIMIT 5;" REP: Thu Mar 19 15:17:59 2026
id      serial_number  timestamp      temperature    humidity      pressure      light    gas    created_at
2710    STATION01      2026-03-19 15:17:58  25.3999 0      993.631 9.16667 1958 2026-03-19 14:17:58
2709    STATION01      2026-03-19 15:17:55  25.3977 0      993.624 9.16667 1958 2026-03-19 14:17:55
2708    STATION01      2026-03-19 15:17:53  25.3951 0      993.626 9.16667 1947 2026-03-19 14:17:53
2707    STATION01      2026-03-19 15:17:51  25.3973 0      993.617 9.16667 1960 2026-03-19 14:17:51
2706    STATION01      2026-03-19 15:17:49  25.3928 0      993.641 9.16667 1959 2026-03-19 14:17:49
```

L6 — Backup Server

Maksym Bylym

Table of Content

1. Overview	4
2. Configure Web/DB Server (L4) for Remote Access.....	4
2.1 Expose MySQL Port in Docker.....	4
WEB/DB SERVER	4
2.2 Create MySQL Backup User	4
WEB/DB SERVER	4
3. Create and Configure the Backup Server.....	5
3.1 Create the Virtual Machine.....	5
BACKUP SERVER.....	5
3.2 Install Ubuntu Server	5
BACKUP SERVER.....	5
3.3 Initial System Configuration.....	5
BACKUP SERVER.....	5
3.4 Configure Static IP Address.....	6
BACKUP SERVER.....	6
4. Backup Directory and Shell Script.....	6
4.1 Create Backup Directory	6
BACKUP SERVER.....	6
4.2 Store Credentials Securely (mysql_config_editor)	7
BACKUP SERVER.....	7
4.3 Create the Backup Shell Script.....	7
BACKUP SERVER.....	7
5. Automate Backups with Cron	8
5.1 Configure Cron Job.....	8
BACKUP SERVER.....	8
6. FTP Access to Backup Directory	9
6.1 Install vsftpd.....	9
6.2 Configure vsftpd.....	9
BACKUP SERVER.....	9
6.3 Create FTP User	10
BACKUP SERVER.....	10
7. Manual Database Restore Procedure	10

WEB/DB SERVER	10
8. Full OS Backup with rsync	11
8.1 Install rsync and Create OS Backup Directory.....	11
BACKUP SERVER.....	11
8.2 Create OS Backup Script	11
BACKUP SERVER.....	11
9. Requirements Checklist	12
9.1 Must.....	12
9.2 Should.....	12
9.3 Could.....	12

1. Overview

In this sprint I set up a dedicated backup server for the PIF project. The goal was to automatically back up the MariaDB project database (pif_db) from the existing web/DB server (L4) to a separate virtual machine every day at 08:00, and to provide FTP access to the backup directory so backup files can be retrieved over the network.

The solution builds on the infrastructure from L4: the same Docker-based MariaDB container (pif-database) on the Ubuntu Server (REPIF) is used as the source. A new Ubuntu Server VM (bylma731-backup) was created as the backup target. A shell script using mysqldump performs the database backup; vsftpd provides FTP access; and cron automates the schedule. An OS-level backup using rsync was also configured.

2. Configure Web/DB Server (L4) for Remote Access

These steps were performed on the existing L4 server (REPIF) where Docker and pif-database are running.

2.1 Expose MySQL Port in Docker

WEB/DB SERVER

MySQL inside the Docker container already listens on port 3306 internally. To allow the backup server to connect, the port must be published externally in dockercompose.yml:

```
nano ~/pif-server/docker-compose.yml
```

In the database service section, add the port mapping:

```
database:  image: mariadb:11  ports:  -  
"3306:3306"
```

Save the file and restart the containers:

```
cd ~/pif-server docker  
compose down docker compose up -d
```

Verify the port is published:

```
docker compose ps  
# database row should show: 0.0.0.0:3306->3306/tcp
```

2.2 Create MySQL Backup User

WEB/DB SERVER

A dedicated MySQL user was created with only the minimum privileges required for mysqldump. Connect to the MariaDB container:

```
docker exec -it pif-database mariadb -u root -p
# Password: rootpassword
```

Run the following SQL commands (replace 192.168.X.X with the backup server IP):

```
CREATE USER 'backup_user'@'192.168.X.X'
  IDENTIFIED BY 'backup_password';

GRANT SELECT, LOCK TABLES, SHOW VIEW,
EVENT, TRIGGER ON pif_db.*
  TO 'backup_user'@'192.168.X.X';

FLUSH PRIVILEGES;

-- Verify the user was created:
SELECT User, Host FROM mysql.user
  WHERE User = 'backup_user';
EXIT;
```

The privileges SELECT, LOCK TABLES, SHOW VIEW, EVENT, and TRIGGER are the minimum required for mysqldump. This satisfies the Should requirement for least-privilege access.

3. Create and Configure the Backup Server

All steps below were performed on the new VM (bylma731-backup).

3.1 Create the Virtual Machine

BACKUP SERVER

- VM name: bylma731-backup
- OS: Ubuntu Server 24.04 LTS
- Network: Bridged Adapter

3.2 Install Ubuntu Server

BACKUP SERVER

- Install OpenSSH server option selected during setup
- Username: bylma731

3.3 Initial System Configuration

BACKUP SERVER

Update the system:

```
sudo apt update && sudo apt upgrade -y
```

Set the correct timezone:


```
sudo timedatectl set-timezone Europe/Luxembourg timedatectl  
# Verify timezone is correct
```

Install the MySQL client software:

```
sudo apt install -y mysql-client sudo  
apt install -y mariadb-client
```

3.4 Configure Static IP Address

BACKUP SERVER

Identify the network interface name:

```
ip link show  
#enp0s3
```

Open the Netplan configuration file:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Replace the contents with the following (substitute the values assigned by the teacher):

```
network:  
version: 2  
ethernets:  
  enp0s3:  
    dhcp4: no  
    addresses: [192.168.X.Y/24]  
    routes:  
      - to: default  
        via: 192.168.X.1  
    nameservers:  
      addresses: [8.8.8.8, 8.8.4.4]
```

Apply the changes and verify:

```
sudo netplan apply ip  
addr show
```

4. Backup Directory and Shell Script

4.1 Create Backup Directory

BACKUP SERVER

Create the backup directory structure and set permissions:

```
sudo chown bylma731:bylma731 /backup sudo  
chmod 755 /backup sudo chown  
sudo mkdir -p /backup/mysql bylma731:bylma731 /backup/mysql sudo  
chmod  
700 /backup/mysql
```

Permission 700 on /backup/mysql means only the owner can read and write. The FTP user is granted access separately in a later step.

4.2 Store Credentials Securely (mysql_config_editor)

BACKUP SERVER

To avoid storing the database password in plain text inside the shell script, the `mysql_config_editor` utility is used. It stores credentials in an encrypted file (`~/.mylogin.cnf`):

```
mysql_config_editor set \  --login-path=backup \  
    --host=192.168.X.X \  
    --user=backup_user \  
    --password  
#password: backup_password
```

Verify the configuration was saved:

```
mysql_config_editor print --all
```

This satisfies the Should requirement: the MySQL user password is not stored in the shell script.

4.3 Create the Backup Shell Script

BACKUP SERVER

Create the shell script file:

```
nano /backup/backup_mysql.sh
```

Script contents:

```
#!/bin/bash  
  
# Configuration  
DB_NAME="pif_db"  
BACKUP_DIR="/backup/mysql"  
DATE=$(date +%Y-%m-%d_%H-%M-%S)  
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_${DATE}.sql"  
LOG_FILE="/backup/backup.log"
```

```

RETENTION_DAYS=7

# Logging function log()
{
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}

log "INFO: Starting backup of $DB_NAME"

# Perform the backup using --login-path (no plain-text password)
mysqldump --login-path=backup \
    --single-transaction \
    --routines \
    --triggers \
    "$DB_NAME" > "$BACKUP_FILE" 2>> "$LOG_FILE"

# Check result if [ $? -eq 0 ]; then    log
"SUCCESS: Backup saved to $BACKUP_FILE" else    log
"ERROR: Backup failed!"    exit 1 fi

# Delete backup files older than 7 days
find "$BACKUP_DIR" -name "*.sql" \    mtime
+$RETENTION_DAYS -delete log
"INFO: Old backups cleaned up"

```

Save the file, then make it executable:

```
chmod +x /backup/backup_mysql.sh
```

Run the script manually to test it:

```

bash /backup/backup_mysql.sh ls
-ls /backup/mysql/
# Verify that a .sql file was created cat
/backup/backup.log # Verify that SUCCESS is logged

```

5. Automate Backups with Cron

5.1 Configure Cron Job

BACKUP SERVER

Open the crontab editor for the current user:

```
crontab -e  
# Select nano
```

Add the following line at the end of the file:

```
# Back up the database every day at 08:00  
0 8 * * * /backup/backup_mysql.sh
```

Verify the cron entry was saved:

```
crontab -l
```

Ensure the cron daemon is running and enabled at boot:

```
sudo systemctl status cron sudo  
systemctl enable cron
```

*Cron format: minutes hours day-of-month month day-of-week. The expression 0 8 * * * means every day at 08:00.*

6. FTP Access to Backup Directory

6.1 Install vsftpd

BACKUP SERVER

```
sudo apt install -y vsftpd
```

6.2 Configure vsftpd

BACKUP SERVER

Back up the original configuration file first:

```
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak
```

Edit the configuration:

```
sudo nano /etc/vsftpd.conf
```

Ensure the following parameters are set (add them if missing):

```
anonymous_enable=NO local_enable=YES  
write_enable=YES chroot_local_user=YES  
# FTP jail - user cannot leave  
/backup/mysql allow_writeable_chroot=YES  
local_root=/backup/mysql pasv_enable=YES  
pasv_min_port=40000 pasv_max_port=50000
```

chroot_local_user=YES confines the FTP user to /backup/mysql so they cannot navigate to other parts of the filesystem. This satisfies the Could requirement for FTP jail.

6.3 Create FTP User

BACKUP SERVER

Create a system user for FTP access without shell login:

```
sudo adduser ftpuser --shell /usr/sbin/nologin
# Set a password for the FTP user when prompted
```

Grant the FTP user access to the backup directory:

```
sudo chown -R ftpuser:ftpuser /backup/mysql sudo
chmod 755 /backup/mysql
```

Start vsftpd and enable it at boot:

```
sudo systemctl restart vsftpd
sudo systemctl enable vsftpd sudo systemctl status
vsftpd
```

Test FTP access from any machine on the network:

```
ftp 192.168.X.Y #
Username: ftpuser
# Password: [your password] ls
# Should list the backup .sql files
```

7. Manual Database Restore Procedure

The following procedure describes how to manually restore the project database from a backup file. These commands are run on the web/DB server (REPIF).

WEB/DB SERVER

Step 1: Copy the backup file from the backup server to the DB server:

```
scp bylma731@192.168.X.Y:/backup/mysql/pif_db_2026-01-01.sql ~/
```

Step 2: Import the backup into the MariaDB Docker container:

```
docker exec -i pif-database mariadb \
-u root -prootpassword pif_db \
< ~/pif_db_2026-01-01.sql
```

Step 3: Verify the data was restored:

```
docker exec -it pif-database mariadb \  
-u root -p pif_db \  
-e 'SHOW TABLES; SELECT COUNT(*) FROM measurements;'
```

This documented restore procedure satisfies the Should requirement for a manual import procedure.

8. Full OS Backup with rsync

In addition to the database backup, a complete backup of operating system files was configured using rsync. This allows recovery in the event of a VM or hardware crash.

8.1 Install rsync and Create OS Backup Directory

BACKUP SERVER

```
sudo apt install -y rsync sudo mkdir -p  
/backup/os sudo chown bylma731:bylma731 /backup/os
```

8.2 Create OS Backup Script

BACKUP SERVER

```
nano /backup/backup_os.sh
```

Script contents:

```
#!/bin/bash  
LOG_FILE="/backup/backup.log"  
DATE=$(date +%Y-%m-%d_%H-%M-%S)  
  
log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"  
>> "$LOG_FILE"  
}  
  
log "INFO: Starting OS backup"  
  
sudo rsync -aAX --delete \  
--exclude={"/proc/*","/sys/*","/tmp/*","/run/*",  
"/mnt/*","/dev/*","/backup/*"} \  
/ /backup/os/ 2>> "$LOG_FILE"  
  
if [ $? -eq 0 ]; then log  
"SUCCESS: OS backup completed"  
else log "ERROR: OS backup  
failed" fi
```

```
chmod +x /backup/backup_os.sh
```

Add the OS backup to cron (runs every Sunday at 02:00):

```
crontab -e  
  
0 2 * * 0 /backup/backup_os.sh
```

9. Requirements Checklist

9.1 Must

- Backup server installed and configured (bylma731-backup, Ubuntu Server 24.04)
- Daily database backup at 08:00 via cron (/backup/backup_mysql.sh)
- FTP access to backup directory configured (vsftpd, ftpuser, /backup/mysql)
- Documentation: this document covers all installation and configuration steps

9.2 Should

- MySQL password not in shell script — stored via mysql_config_editor (~/.mylogin.cnf)
- Backup user (backup_user) has minimum required privileges only
- Manual restore procedure documented in Step 6
- Full OS backup via rsync configured and scheduled weekly

9.3 Could

- Backup files older than 7 days automatically deleted (find -mtime +7)
- FTP jail configured (chroot_local_user=YES in vsftpd.conf)
- Successful and failed backups logged to /backup/backup.log